TECHNICAL REFERENCE

Power Glove Vision Configuration Reference

Active files, installed copies, fields, secrets, and generated state.

2026 EDITION / IAIN BENNETT / MIT LICENSE

Use this reference to find a setting, change a game mapping, tune a gesture, or look up a command. Each section identifies the active file and explains what its values mean.

Use the installation guide for the initial deployment and pairing procedure. Return here when you need to change a host, camera, game, gesture threshold, or network setting.

In this guide, **PowerGlove Vision Controller** means the Arduino UNO Q running the camera, recognition, dashboard, and controller-sending software. Hardware- specific commands and filenames retain `uno-q` where that literal name is required.

> **KEEP THE TOKEN PRIVATE** PowerGlove Vision uses one shared token to authenticate controller packets and profile changes. Never paste it into an issue, screenshot, command line, public backup, or Git commit.

The three places configuration lives

PowerGlove Vision runs on two computers. A repository template is not always the file the running system reads.

Location	What it controls	Preferred way to change it
PowerGlove Vision Controller Setup page	Console address, controller port, startup profile, camera, and pairing token	Browser Setup page
PowerGlove Vision Controller application files	Gesture sensitivity and advanced runtime defaults	Edit only when tuning is required
RetroPie `/etc/powerglove/`	PowerGlove Vision Controller address, per-game profile selection, and receiver token	Protected files on RetroPie

The examples under the repository's `config/` directory are installation templates. Editing them does not change an already installed system. The active copies are identified in each section below.

In commands and examples, replace these placeholders:

Placeholder	Replace with
`UNO-Q-NAME.local`	Your PowerGlove Vision Controller hostname or reserved IP address
`RETROPIE-NAME.local`	Your RetroPie hostname or reserved IP address
`/home/arduino/ArduinoApps/powerglove-vision`	Required directory for the supported PowerGlove Vision Controller installer and shutdown helper; do not substitute a different path

Find the setting or command you need

Task	Section
Install and pair both machines	Installation Guide
Change the camera or startup profile	PowerGlove Vision Controller settings
Repair pairing or token permissions	Pairing and token management
Change the PowerGlove Vision Controller destination on RetroPie	RetroPie connection settings
Make a game select a profile	Register games and select profiles
Adjust gesture sensitivity	Tune gesture sensitivity
Understand an option or command	Command-line reference

PowerGlove Vision Controller settings

Open the ordinary Setup page at:

```
http://UNO-Q-NAME.local:8088/setup
```

Pairing uses the secure Setup page instead:

```
https://UNO-Q-NAME.local:8443/setup
```

The secure page uses a locally generated certificate. During pairing, compare the certificate fingerprint shown by the browser with the identifier shown on the PowerGlove Vision Controller matrix before entering the one-time PIN.

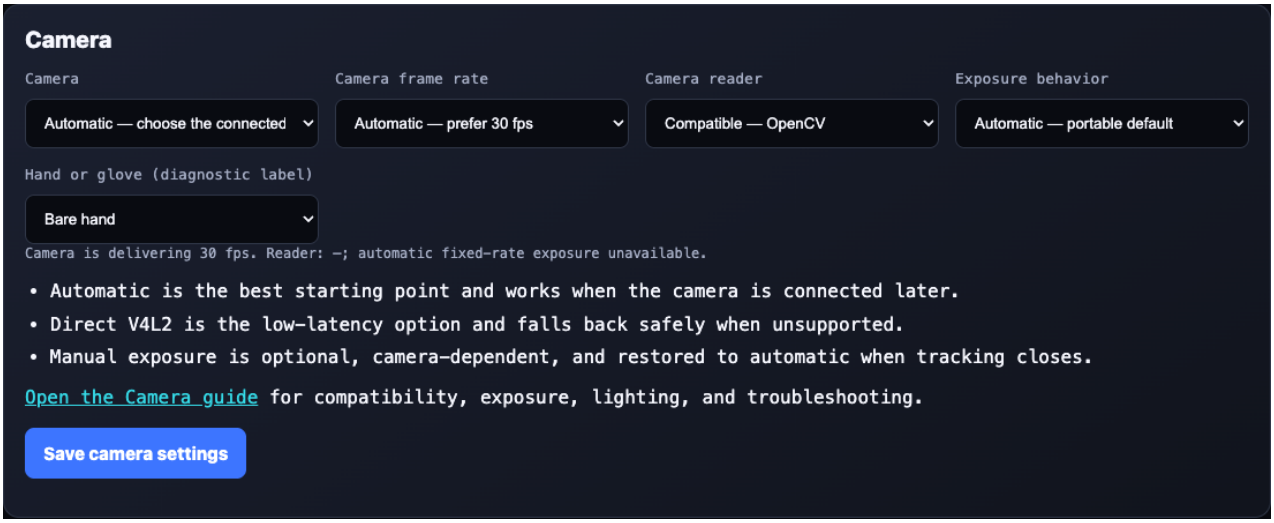
Settings shown in the browser

Setup groups **Controller status**, **Players**, **Matrix attract mode**, **Connection and startup**, **Camera**, **Pair with RetroPie**, **Games**, and **Show statistics**. Receiver port and key replacement are under **Advanced connection**. Camera selection, rate, reader, exposure, and diagnostic hand label are in their own action-first Camera section. The concise Camera guide explains compatibility, lighting, and recovery. Key replacement stops output and requires pairing again. A saved destination and key are not proof that RetroPie has received that key. **Check console address** verifies name resolution only.

A failed initial load offers **Reload saved settings**; connection fields remain disabled until loading succeeds. Failed actions can be retried. Start/Stop and pairing refreshes retain unsaved connection edits. HTTP 202 from `/api/controller` means the latest request is saved but still awaiting worker delivery. The supervisor retries it; another explicit request supersedes it. This is separate from camera frames and controller packets, which remain newest-state-only.

Setting	Default	Meaning and recommendation
Console hostname or IP	Empty (not configured)	Set your RetroPie hostname (`RETROPIE-NAME.local` in examples) or a reserved LAN address and pair through Connection before starting controls. Glove Academy and local settings work without a destination. Existing saved destinations are preserved.

Setting	Default	Meaning and recommendation
Receiver UDP port	`55355`	PowerGlove Vision Controller to RetroPie controller-state port. Leave it at the default unless both ends are changed.
Startup game profile	`bad_street_brawler`	Profile used before a registered game selects another one.
Hand or glove (diagnostic label)	`none`	`none`, `white`, or `black`. In the current release this is an informational diagnostic label; it does not change MediaPipe tracking.
Camera	Automatic	Setup lists the currently discovered usable cameras. Prefer **Automatic - choose the connected camera**; choose a named camera only when more than one is attached or automatic selection is wrong. A saved disconnected camera remains visible as unavailable, and the list refreshes while Setup is open.
Camera frame rate	Automatic	Tries 30 fps first, then accepts the camera driver's usable rate if necessary. Explicit 30- and 60-fps requests are available for comparison and fall back safely when unsupported. The live negotiated rate appears below the setting while tracking is active.
Camera reader	Compatible - OpenCV	The normal portable capture path. **Low latency - Direct V4L2** is an opt-in Linux 64-bit, 640×480 MJPEG experiment that drains to the newest driver buffer and falls back to OpenCV if its requirements are not met.
Exposure behavior	Automatic - no camera changes	Leave cameras untouched by default. **Low latency - standard UVC** keeps automatic exposure and requests fixed frame rate only when those controls are advertised. **Razer Kiyo Pro - tested low latency** adds the Kiyo's volatile HDR-off request. **Manual exposure and gain** is available with Direct V4L2 after capability and range checks.
Replace the pairing key when saving	Off	Rotates the shared secret. This immediately breaks the existing pairing until RetroPie is paired again.



Advanced camera settings showing the discovered-camera dropdown and exposure controls

Selecting **Save connection and startup** or **Save camera settings** validates the complete configuration, writes it atomically with private permissions, and restarts the vision worker using the saved calibration. Recalibrate only if you have moved the camera, changed your playing position, or notice unwanted movement while your hand is at rest.

Setup pairing uses the saved console address, with one active step at a time: choose a method, confirm the Controller certificate and matrix approval PIN, then provide the RetroPie one-time code or SSH credentials. Unsaved settings block

pairing. The existing two-minute authorization window and server attempt limits remain authoritative; changing the console or method is disabled during the active window. Expiry clears secrets and offers fresh confirmation. A failed submitted request also requires confirmation again. Password entry is disabled until the certificate comparison and six-digit PIN step is complete. Ordinary HTTP shows only a link to secure Setup. Start/Stop and shutdown remain on Dashboard. See the pairing walkthrough.

When a submitted pairing attempt finishes, the matrix releases the approval PIN and resumes its normal display. When idle, the glove animation follows your On, Dim, or Off attract setting; active game and status displays still take priority.

The Dashboard profile selector changes only the current active profile. It does not rewrite `device.json` or change the Setup page's startup profile. RetroPie may replace a Dashboard selection when a game starts or ends.

Show statistics is a browser-local Dashboard preference, also available at the bottom of Setup. It is off by default and is stored in browser local storage, not `device.json`. When disabled, the Dashboard still polls basic state needed for controls and connection feedback but does not read, render, or retain the optional controller, axes, finger, performance, and recent-event fields. The worker publishes changed controller state immediately. While statistics are shown, routine detailed feedback is refreshed at about 10 Hz and replaces any older pending Dashboard work in a one-item worker queue. Expensive rolling percentiles are refreshed at 2 Hz. With statistics hidden, those calculations are not performed; camera tracking and controller transmission are unchanged.

Comfortable movement range

Native X/Y can map a player's comfortable left, right, up, and down positions to the screen edges. It changes sensitivity and physical travel, not processing time. Gesture thresholds, D-pad behavior, depth, and Latest-coordinate movement stay unchanged.

The four optional `calibration.neutral` fields `reach_left`, `reach_right`, `reach_up`, and `reach_down` are normalized image distances from the saved center. All four zero (or omitted in older backups) use the original camera-boundary mapping. Otherwise all four must be finite numbers at least `0.05` and fit inside the image around that center. Player presets and version-2 hand-setup backups preserve them. New reach-bearing backups require reach-aware software on import. **Center hand preserves valid reach spans** because neutral centering and comfortable travel are separate adjustments. If a new center would place an existing endpoint outside the camera image, the software safely returns to the full-field mapping. Review reach after moving the camera or changing playing position. Never reuse another camera setup's spans as universal defaults.

For normal adjustment, open **Glove Academy -> Tune gestures -> Movement reach**. The four fields load the active player's exact saved spans, and the read-only summary shows the resulting width, height, and aspect ratio. Smaller values need less hand travel. **Save reach values** replaces only these four calibration fields. **Restore full camera field** writes four zeros after confirmation and does not alter center, scale, roll, jitter, thresholds, or Academy progress. Controller output remains paused while Tune gestures is open.

The operator helper `scripts/calibrate-reach.py` runs inside the Controller container, using the worker's Python environment and loopback APIs. After normal player centering, run these as separate commands, cueing the player before each:

```sh
python scripts/calibrate-reach.py begin
python scripts/calibrate-reach.py center
python scripts/calibrate-reach.py left
python scripts/calibrate-reach.py right
python scripts/calibrate-reach.py up
python scripts/calibrate-reach.py down
python scripts/calibrate-reach.py apply
```

`begin` saves a private complete backup under `data/backups/reach-*/`, persistently pauses delivery, and leases practice mode. Hold a relaxed palm at center for `center`, then a steady, comfortable endpoint for each three-second directional step. The helper samples raw palm positions and requires at least 12 independent reliable observations, at least 80% reliable observations, and a stable hold away from the image boundary. Failed holds can be repeated. Do not launch a game, change players, or enter gesture tuning during the session.

`apply` uses the existing generation-checked player restore path. `cancel` restores the original complete hand setup. Both leave output paused for verification; restart delivery explicitly when ready. A pending restore or player change stops the helper rather than applying to a different player; the saved backup remains available in Setup -> Players. Only numeric calibration data is stored, not images.

Worker status exposes raw `palm_position` (`x` and `y`, or `null` when undetected) for this calibration. Its coordinates follow the configured mirror convention. The helper accepts synchronous practice observations only.

Native X/Y movement

The Controller always uses each newest completed MediaPipe palm observation as the native X/Y source. **Latest coordinate** publishes the newest mapped MediaPipe coordinate without temporal smoothing during continuous tracking. There is no movement-mode control on the Dashboard.

The live path validates landmark geometry and clamps the selected point to the active player's reach rectangle before mapping. Movement beyond an edge stays pinned to that edge and cannot build hidden off-screen state. The selected frame's capture timestamp drives freshness; inference-start time is not substituted for it. The production anchor remains the five-point average of wrist and four knuckles so existing centers and reach spans remain valid.

MediaPipe still supplies fingers, depth, roll, and gestures; FCEUmm directions and every game mapping are unchanged. Worker status reports `native_xy_source` as `mediapipe` while the native path is active or `inactive` otherwise. It reports `native_xy_mode` as `latest` for compatibility with existing diagnostics.

The former optical-flow prototype remains in `src/powerglove_vision/motion.py` for historical comparison but is not connected to the live supervisor. Older `motion_tracking` and `native_xy_mode` configuration values are ignored. The hidden `--motion-tracking` command-line spelling remains accepted only so older launch scripts do not fail; it does not enable optical flow. Historical flow diagnostics and trace readers remain useful for analyzing already-recorded experiments.

When MediaPipe misses the hand briefly, the engine holds the last native X/Y for up to the configured `loss_release_ms` (120 ms by default) to avoid an edge departure/re-entry jump. Buttons, fingers, Z, roll, and D-pad state release immediately during that hold. Continued loss, stale data, calibration changes, or profile changes neutralize native X/Y and clear retained coordinate history. On recovery, Latest normally accepts the first fresh coordinate. If established motion is followed by one contradictory result, or the new point is unusually distant without being strongly aligned forward, it holds the last reliable X/Y for one fresh result. The next measurement is authoritative. This guard never predicts a position, modifies continuous tracking, or delays strongly aligned forward recovery.

Selected camera and inference settings

The current baseline uses the complete 640×480 MJPEG image, four MediaPipe inference threads, a `0.35` tracking-confidence threshold, a `2.25` next-frame hand search area, and Automatic camera rate. Automatic tries 30 fps first because live play felt smoother and more attached to the hand than the 60-fps request, then reopens the camera without a forced rate if it cannot produce frames. Setup can explicitly request 30 or 60 fps for comparison and reports the negotiated rate while tracking is active. An unsupported explicit rate also falls back to the driver's usable choice.

The capture and exposure choices are mostly independent. `camera_backend` is `opencv` or `direct-v4l2`; `camera_exposure` is `auto`, `low-latency`, `kiyo-low-latency`, or `manual`. Manual mode requires Direct V4L2 because its controls are applied to the already-streaming camera descriptor; it never opens a competing second camera

connection. `camera_manual_exposure` and `camera_manual_gain` store the requested whole-number values. Direct V4L2 requires Linux's 64-bit V4L2 ABI and an already negotiated 640×480 MJPEG stream. It exposes the camera sequence and monotonic driver timestamp when the driver supplies them. Any initialization or format failure is reported as `capture_backend_fallback`, then the camera is reopened through OpenCV. No game, gesture, reach, or calibration setting is changed.

Every exposure mode first queries the standard V4L2 controls. Unsupported or disabled controls are never written. Low-latency mode leaves exposure automatic while disabling exposure auto-priority so long exposures cannot silently reduce frame cadence. The Kiyo mode additionally verifies USB identity before sending the existing HDR-off request. Manual mode requires both exposure and gain controls, validates the camera's advertised minimum, maximum, and step, switches exposure to manual, and reads back the applied values. If the request is unsupported, the worker reports an explicit automatic fallback; if a partial write cannot be safely restored, startup fails instead of leaving an unknown camera state. Closing vision restores automatic exposure. These changes are volatile and do not rewrite settings inside the camera.

Automatic remains the portable default. In the September 8 matched live test, this project's Razer Kiyo Pro at manual exposure `78` and gain `96` had slightly better detection continuity than automatic exposure, with indistinguishable latency. Those values are a saved setting for that Controller, not a universal camera preset. Other cameras should begin on Automatic and use Manual only after checking the reported supported range and live hand image.

The earlier Kiyo Pro capture experiment used two buffers and a volatile HDR-off command while requesting 60 fps. It remains useful historical evidence, but it is not the 0.4.0 general default. See the capture comparison.

The general default remains one buffer and no vendor control command. The HDR option checks USB identity `1532:0e05`, sends only the volatile HDR-off command, and sets and verifies automatic exposure with dynamic frame rate disabled. It does not save settings onboard. Other camera models are left untouched. Control failure is reported as `camera_control_error`; capture may continue, but that run must not be treated as a verified candidate. `camera_hdr_off_command_sent` confirms the command was accepted, not a readback of the sensor's HDR state. Buffer metadata reports requested/accepted values separately from real frame-delivery measurements.

Remove these two settings (or set buffers to `1` and HDR-off to `false`) and restart to restore the original software capture policy. The application does not issue an HDR-on command on rollback; power-cycle the camera to reload its saved settings. Camera configuration is separate from player calibration and reach backups.

Vision startup and timing

Camera capture uses the complete 640×480 field of view. Automatic rate prefers 30 fps with safe driver fallback. A dedicated capture thread continuously drains the camera and retains only its newest frame, so inference skips superseded images instead of building an input queue. The deployed worker explicitly selects **MediaPipe Hands**, whose stable command identifier is `legacy`, and lets each of its two inference stages use up to four CPU threads (`--inference-threads 4`). Matched live testing promoted four threads for the PowerGlove Vision Controller; the saved setting remains explicit so a future platform can be re-measured rather than assuming the same result. **MediaPipe Tasks Video (experimental)** remains available under the stable `tasks-video` identifier for controlled comparison.

Camera-preview drawing and JPEG encoding run only while a browser is actively watching the Dashboard or Glove Academy stream. JPEG encoding runs on a separate latest-preview worker and is allowed to drop superseded preview jobs rather than delay another controller sample. During gameplay, normalized landmark drawing and JPEG encoding use a 320×240 copy after inference; Academy and tuning retain the full-size preview. Closing those pages avoids even that optional work. When a preview is open, submissions are limited to 5 fps (`--preview-fps 5`).

The full MediaPipe graph remains the production setting. The configuration-only `lean-image` experiment requests only image landmarks from the same MediaPipe Hands graph, omitting handedness and world-landmark output streams. It does not replace the hand model or reduce the 21 landmarks calculated for recognition. An output-paused Controller comparison improved median inference only from about 43.9 ms to 43.2 ms and worsened p95 sample age from about 108.5 ms to 114.6 ms. That result did not pass the promotion gate, so `full` remains the supported default. Retain

`lean-image` only for repeatable comparisons on future runtimes or hardware.

The worker preloads OpenCV and MediaPipe on its background vision thread as soon as its control server is available. Preloading imports the libraries; it does not open the camera, create a hand tracker, or process images. The website and profile controls remain responsive while imports run. A failed preload is logged; a later activation retries loading and reports any error.

If **Gestures off** is selected at startup, the camera stays closed until you select an active profile or open Glove Academy. An active startup profile requests capture automatically after preloading. The player's explicit controller choice is restored as **armed** or **stopped**, but an armed worker does not transmit until a live registered game session or intentional manual Dashboard profile exists.

Activation waits for any unfinished preload, verifies the saved model, opens and configures the camera, waits for a usable frame, and creates the tracker. Dashboard, Play, and Glove Academy show **Starting camera and gesture tracking** until vision is active. The elapsed time covers startup work, not only the physical camera. **Center hand** stays disabled until initialization finishes.

Switching between active profiles reuses the camera and tracker. **Gestures off** releases both, while imported libraries remain in memory. An application restart, including a worker restart after saving Setup settings, starts preloading again.

Measurements on the cabinet's PowerGlove Vision Controller with a Razer Kiyo Pro on September 4, 2026:

Measurement	Observed time
First activation after reboot, before background preloading	7.67 seconds
OpenCV background preload after a later reboot	0.94 seconds
MediaPipe background preload after that reboot	5.94 seconds
First activation after that preload completed	1.21 seconds

These were separate reboot tests on this cabinet, not guaranteed timings. The change moves library loading earlier; it does not eliminate that work. Selecting a profile immediately after application startup can still wait for preloading. An earlier reported 13-14 second delay was not reproduced in the instrumented tests.

To inspect startup stages on the PowerGlove Vision Controller:

```
SH
docker logs --since 10m powerglove-vision-main-1 2>&1 | grep 'Vision startup:'
```

The default installation uses this container name; use `docker ps` to find it if your App Lab installation uses another name. Logs include background import times, model verification/recovery, camera discovery/open/settings, the first camera frame, tracker construction, first inference, and time to active worker status. Preparation and activation totals include earlier stages; do not add them to the individual durations. Dashboard polling adds a small delay before it displays the new status. Compare entries from the same activation.

If the camera is unavailable, check `lsusb` and `/dev/v4l/by-id/` on the PowerGlove Vision Controller. The built-in `qcom-venus-encoder` and `qcom-venus-decoder` video nodes are not webcams. A camera missing from the USB device list is below MediaPipe and OpenCV; preloading cannot resolve that condition. The installed host helper waits for a sustained outage before making one guarded reset of the camera's last observed parent hub. It disables autosuspend whenever the single UVC camera is present. If that one attempt does not restore USB enumeration, check the powered hub, cable, and camera connection.

Glove Academy, calibration, and live readings

Open **Glove Academy** at `/learn` to practise gestures, calibrate your resting position, or personalize recognition. The matrix shows **L** for lessons and **T** for tuning.

Glove Academy starts the camera even when **Gestures off** is selected and uses a mapping-independent practice profile. It pauses controller delivery and restores the selected profile when you leave. With several Glove Academy tabs open, practice remains active until the last tab closes or its lease expires. A six-second lease timeout handles an unexpected browser close. Loading Dashboard also clears a stale session; reload Glove Academy if you want to begin practice again.

The sixteen lessons cover hand visibility and neutral position, movement, finger curls, Start and Select, forward and backward gestures, wrist roll, closed hand, and Menu Guard. Choose your player before practising; lesson progress is saved for that player across restarts. Complete every lesson to earn **Glove Master**; skipped lessons must be revisited. The award replaces the completed lesson in the same card. **Start again** clears the selected player's lesson progress and award, while retaining their hand settings and calibration.

For hand shapes and illustrations, use the Gameplay Guide's gesture reference. Its game cards explain what each gesture does in a selected profile. Academy practice indicators do not change those mappings.

Recognition readings and calibration controls

Reading or control	Meaning
Finger curl	Glove Academy shows values from 0 to 1; Dashboard uses a compact 0-to-3 display. Default ordinary curl actions engage at 0.50 and release below 0.35; saved personal pairs override these values.
V sign	Without personal adjustments, index and middle curl must be below 0.28; ring and little curl must exceed 0.42. Hold steadily for 0.50 seconds to send Start. A non-V pose must then remain visible for 0.30 seconds before Start can rearm.
Thumbs-up	Without personal adjustments, thumb curl must be below 0.32 and all four finger curls above 0.42. Hold for 0.15 seconds to send Select.
Live hand measurements	Shows curl values, thresholds, enlarged landmarks, and forward or backward movement relative to the calibrated hand size.
Center hand	Replaces the saved resting reference. The button turns red while sampling, then blue with a brief completion message.

Tracking and timing diagnostics

These fields describe recognition and the local processing path. They do not measure the complete delay from physical hand movement to the displayed game.

Diagnostic field	Meaning
`tracker_backend`	Stable command identifier: `legacy` means **MediaPipe Hands**; `tasks-video` means **MediaPipe Tasks Video (experimental)**.
`capture_age_ms`	Time from completion of the newest camera read to the beginning of inference. This is a freshness diagnostic, not the camera exposure timestamp.
`capture_interval_ms`, `capture_skipped_total`	Spacing between processed camera frames and the cumulative number intentionally superseded by newer frames.
`inference_ms`, `inference_interval_ms`, and `inference_hz`	Per-frame tracking calculation, spacing between recognition passes, and its reciprocal rate.
`sample_age_ms`	Time from completion of the newest camera read through inference and the local UDP send attempt. This is the primary PowerGlove Vision Controller software-stage gameplay measurement.
`controller_transition_age_ms`	`sample_age_ms` recorded only when a successfully transmitted gameplay-visible controller state changes. Native X/Y changes are included.
`performance`	Rolling latest, p50, p95, and maximum values over the most recent 300 valid samples for capture age, processed-frame spacing, inference, inference spacing, send time, complete sample age, and changed-control age.
`preview_encode_ms`, `preview_dropped`	Background JPEG cost and previews discarded to protect controller responsiveness.
`send_ms`	Local controller-state send time. None of these readings alone measures the full delay from physical motion to the displayed game frame.

How recognition and calibration behave

Finger recognition uses the strongest joint bend, including the base knuckle; a middle-knuckle bend alone can qualify. Thumb recognition uses the stronger of its two outer joints. MediaPipe 3D world landmarks are preferred. The fallback uses normalized depth with image aspect-ratio correction; palm movement still uses image coordinates. The legacy Arduino landmark bridge keeps its 2D interpretation because its depth units differ.

Glove Academy and gameplay share held finger and movement states. Glove Zap and Pull Back need two consecutive beyond-threshold observations plus 0.10 normalized palm-scale movement in the intended direction within 250 ms. Once confirmed, they remain recognized until movement falls below their respective release thresholds, and a confirmed menu pose still satisfies its lesson after the short controller pulse ends. The browser preview is capped at 5 fps; status updates follow each tracking calculation.

Menu Guard requires curled thumb and ring finger with index, middle, and pinky extended. It suppresses D-pad movement, A, B, Start, and Select, but does not freeze native Super Glove Ball's continuous hand position. Use **Stop controller** when you need to reposition without sending controls.

The app reuses its saved resting reference across Glove Academy, gameplay, profile changes, and restarts. It calibrates automatically only when that reference is missing or invalid. Use **Center hand** after moving the camera or changing your playing position. Keep your palm near the resting position when practising finger curls so unintended movement does not obscure the finger readings. See Saved neutral-hand calibration for storage and recovery details.

Controller delivery and game sessions

Start controller and **Stop controller** set a persistent player choice. Start arms delivery; Stop remains sticky until explicitly changed. Armed is distinct from sending: output also requires either a live registered RetroPie game session or an intentional manual Dashboard profile. The RetroPie hook waits until RetroArch is actually running, then renews a bounded session every two seconds. The PowerGlove Vision Controller accepts each renewal for six seconds and uses a short one-second initialization guard before the first gameplay packet. This avoids operating the runcommand menu while allowing an armed controller to resume automatically if the PowerGlove Vision Controller application restarts during play. Game exit, RetroArch termination, an unregistered launch, or lease expiry sends a neutral/off request. None of these events silently changes a sticky Stop choice.

The Dashboard separates **Controller delivery** from **Game session**. "Armed - waiting for game" means the user's Start choice is retained but no packets are being sent. "Registered game active" identifies a renewable launch session; "Manual profile" identifies an intentional Dashboard testing context.

Active PowerGlove Vision Controller device file

The Setup page maintains this private file inside the application:

```
/home/arduino/ArduinoApps/powerglove-vision/data/device.json
```

A typical device configuration file contains the following fields:

```
JSON
{
  "receiver": "RETROPIE-NAME.local",
  "port": 55355,
  "token": "private-random-value-created-by-the-application",
  "profile": "bad_street_brawler",
  "glove_color": "none",
  "camera": "auto",
  "camera_fps": "auto",
  "inference_threads": 4,
  "tracking_confidence": 0.35,
  "tracking_roi_scale": 2.25,
  "matrix_attract": "on"
}
```

`camera_fps` is `auto`, `30`, or `60`; Automatic prefers 30 and then accepts a usable driver rate. `inference_threads` accepts 1, 2, or 4. The 0.4.0 baseline uses four threads, `tracking_confidence` 0.35, and `tracking_roi_scale` 2.25. Latest coordinate is the only live native X/Y behavior. Older `native_xy_mode` values are accepted in existing files but ignored.

`matrix_attract` accepts `on` (default), `dim` (animation limited to levels 1-2), or `off` (four faint app/console/paired-console/Networking indicators). Change it using **Setup -> Matrix attract mode**. This writes the private device configuration without restarting vision or changing controller state. Existing files that omit it retain the original animation. The separate guarded `POST /api/attract` accepts `{"mode":"on"}`, `dim`, or `off`, with JSON content type and the same-origin `X-PowerGlove-Action: attract` header. Connection indicators use the existing authenticated Games service in a background thread; see the Matrix guide for their meaning and refresh interval. Updated matrix firmware is required.

Use the Setup page for routine changes. If you must edit the JSON directly, stop the application first, keep the token unchanged, validate the file, and restore mode `0600`. JSON does not allow comments or trailing commas.

```
SH
python3 -m json.tool data/device.json >/dev/null
chmod 0600 data/device.json
```

Deleting `device.json` causes PowerGlove Vision to create a new token and first-run defaults. You must then pair RetroPie again.

Camera selection

`auto` searches stable `/dev/v4l/by-id/` capture-device links first, ignores known codec-only video nodes, and then considers ordinary `/dev/video*` capture devices. This is the most reliable choice when USB enumeration changes after a reboot.

Choose a named camera from Setup only for troubleshooting or when more than one camera is connected. The saved value remains the device's numeric V4L2 index for compatibility, but ordinary users do not need to type or discover that number. Linux may assign a different number after hardware is reconnected. Keep the camera on a powered USB hub when the PowerGlove Vision Controller cannot supply stable power by itself.

Supported startup profiles

The valid startup profile identifiers are:

```
off
bad_street_brawler
super_glove_ball
program_a  program_b  program_c  program_d  program_e
program_f  program_g  program_h  program_i
```

The startup profile does not assign a profile to a ROM. Per-game selection is controlled by the RetroPie game registry described below.

Pairing and token management

Both machines must hold the same token:

Machine	Active private file
PowerGlove Vision Controller	Application `data/device.json`, in the `token` field
RetroPie	`/etc/powerglove/token`

Use one-time-code pairing whenever possible:

```
SH
sudo /opt/powerglove/bin/powerglove-pair
```

Leave that command running on RetroPie, then complete pairing at `https://UNO-Q-NAME.local:8443/setup`. The code is single use and expires after two minutes. Password pairing is also available when RetroPie accepts SSH password login; the password is used for one encrypted operation and is not stored.

The RetroPie token must contain at least 16 characters and should remain owned by `root`, readable by the `input` group, and inaccessible to other users:

```
SH
sudo chown root:input /etc/powerglove/token
sudo chmod 0640 /etc/powerglove/token
```

Recover without browser pairing

Use this fallback only when neither browser pairing method works. Both machines must already have the software installed.

1. In App Lab, open the active application's private `data/device.json` and locate its `token` value.
2. On RetroPie, run `sudo nano /etc/powerglove/token`. Replace the file contents with that same value on one line, without quotation marks. Do not enter it as a shell command.
3. Save with Ctrl+O, confirm the filename, and exit with Ctrl+X. Apply the ownership and permission commands above.
4. Run `sudo systemctl restart powerglove-receiver.service`, then test controller delivery from Dashboard. Clear the token from your clipboard and close the private file afterward.

If you replace the pairing key in Setup, controller output stops; pair the devices again before selecting Start controller. Do not transfer the new token through a command-line argument; process listings and shell history can expose it.

RetroPie connection settings

The active launcher file is:

```
/etc/powerglove/launcher.json
```

It tells the runcommand hooks where to send profile changes when a game starts or exits.

```
JSON
{
  "uno_q": "UNO-Q-NAME.local",
  "port": 55356,
  "token_file": "/etc/powerglove/token",
  "registry": "/etc/powerglove/games.json",
  "timeout": 0.4
}
```

Field	Meaning
`uno_q`	PowerGlove Vision Controller hostname or reserved address reachable from RetroPie.
`port`	RetroPie to PowerGlove Vision Controller profile-control port. This is `55356`, not the controller-state port.
`token_file`	Protected shared-token file. Keep the token out of this JSON file.
`registry`	Active ROM-to-profile mapping.
`timeout`	Seconds to wait for each acknowledgement. The hook retries up to three times and never prevents a game from launching.

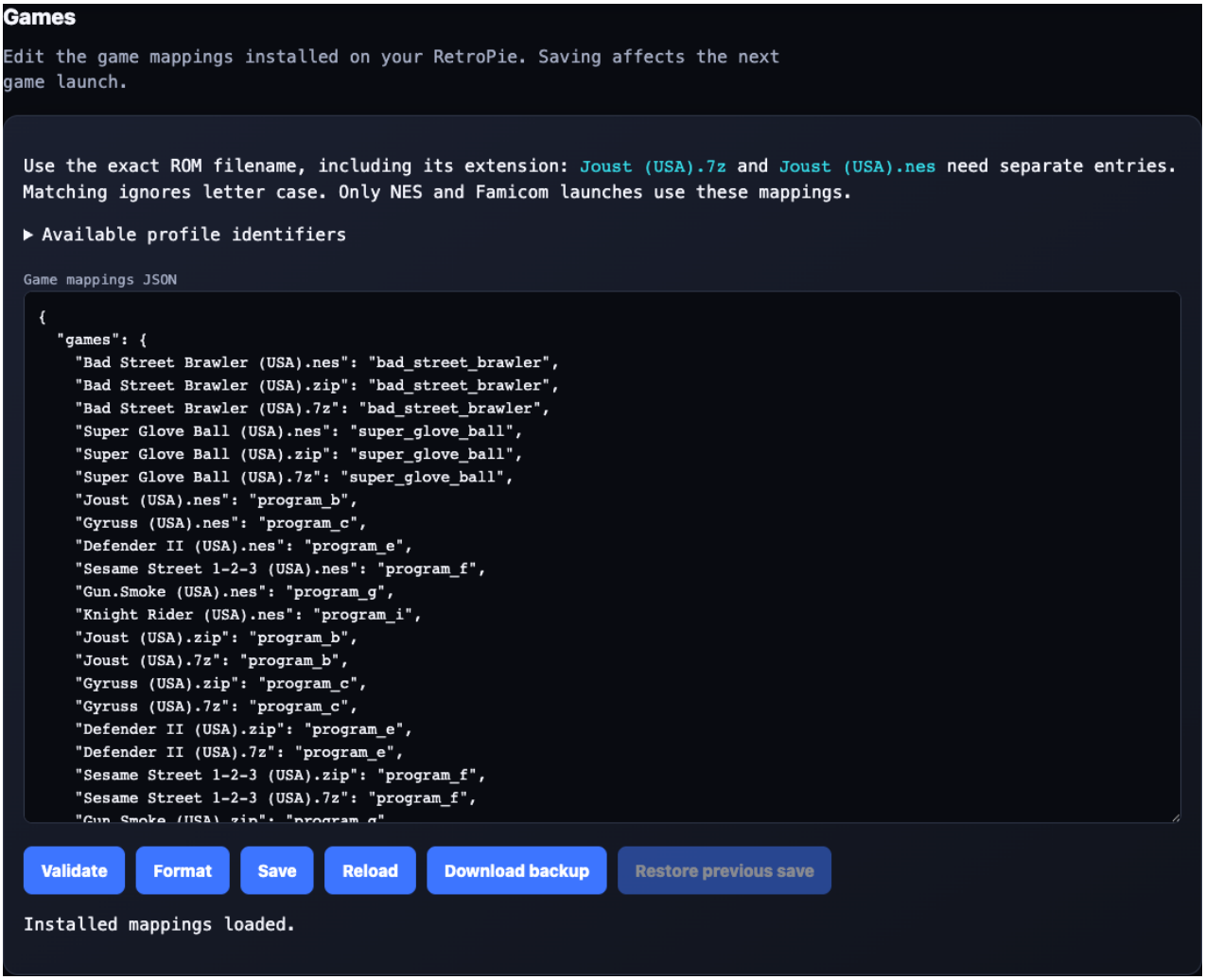
Validate changes before launching a game:

```
SH
python3 -m json.tool /etc/powerglove/launcher.json >/dev/null
```

Use a `.local` hostname when multicast DNS is reliable on your LAN. A reserved DHCP address is a useful fallback. Do not use an address that may later be assigned to another device.

Register games and select profiles

Edit mappings in Games



Games section below pairing on Setup

Games is a section of **Setup**, below pairing; it is not a separate navigation tab.

1. Open **Setup -> Games** in the PowerGlove Vision Controller website. Both machines must be online and paired. The page reads the registry used by the installed RetroPie launch hook.
2. Select **Download backup** to keep a copy of the last verified installed registry on your computer.
3. Edit the JSON, adding the exact ROM filename and a supported profile identifier inside `games`. Expand **Available profile identifiers** for the choices. Preserve your existing entries.
4. Select **Validate**. It checks JSON syntax, supported profiles, and duplicate filenames, including names that differ only by letter case. **Format** tidies the JSON without saving it.
5. Select **Save**. Wait for confirmation that RetroPie saved the file and the PowerGlove Vision Controller read it back successfully.
6. Launch or restart the game and confirm its profile on Dashboard.

Saving does not change the current game's profile. **Restore previous save** swaps in the last valid version. **Reload** discards your draft after confirmation. If another editor changed the installed registry, saving is refused; download or copy your draft before reloading. Connection failures leave the draft in the browser. Leaving or refreshing the page can discard unsaved work.

The Games section needs the `powerglove-games.service` installed by the current RetroPie setup workflow. If it reports an unavailable service, update the RetroPie installation, check pairing, and ensure TCP `55358` is reachable from the PowerGlove Vision Controller. Games does not require an SSH password after pairing.

Registry format

The active game registry is:

```
/etc/powerglove/games.json
```

PowerGlove Vision matches the exact ROM basename, including its extension, without regard to letter case. Directory names are ignored. Automatic profile selection currently applies only when RetroPie reports the system as `nes` or `famicom`; other systems turn gesture control off.

```
JSON
{
  "games": {
    "Bad Street Brawler (USA).nes": "bad_street_brawler",
    "Super Glove Ball (USA).zip": "super_glove_ball",
    "Joust (USA).nes": "program_b"
  }
}
```

Add the exact filename shown in EmulationStation. Zipped and 7-Zip copies need their own entries because `.nes`, `.zip`, and `.7z` are different basenames.

Included game	Profile
Bad Street Brawler	`bad_street_brawler`
Super Glove Ball	`super_glove_ball`
Joust	`program_b`
Gyruss	`program_c`
Defender II	`program_e`
Sesame Street 1-2-3	`program_f`
Gun Smoke	`program_g`
Knight Rider	`program_i`

The table above is the complete set of games recognized automatically by the shipped registry. Programs A, D, and H are fully implemented profiles rather than omitted games: `program_a` is a pinball control scheme, `program_d` reverses all four directions for challenge or accessibility use, and `program_h` provides general-purpose movement with pulsed buttons. They deliberately have no default ROM assignment.

Any appropriate NES or Famicom ROM can use one of those profiles after you add its exact basename to the `games` object. Every profile value must be one of the supported identifiers; an unknown value invalidates the registry.

```
SH
sudo python3 -m json.tool /etc/powerglove/games.json >/dev/null
```

Test profile communication independently of a game:

```
SH
sudo /opt/powerglove/bin/powerglove-profile \
  --uno-q UNO-Q-NAME.local \
  --token-file /etc/powerglove/token \
  --profile program_b
```

Use `--profile off` to stop gesture output. A successful request prints an acknowledgement and starts the matrix glove attract animation. In this healthy idle state the camera and MediaPipe tracker are closed, while the website and authenticated profile listener remain available.

Check a queued profile change

The launch helper reports **profile queued** when the PowerGlove Vision Controller has authenticated and queued the request. Dashboard shows the applied profile and ROM name once the worker processes it. Camera startup can take longer than the acknowledgement; an acknowledgement does not mean that the camera is ready or that controller delivery is enabled.

App Lab's `local:profile_control` brick publishes UDP `55356` through a small relay to the worker. The relay holds no pairing token and forwards signed packets unchanged. Both this brick and `local:avahi_resolver` must be present in `app.yaml`, so the services return when App Lab regenerates its containers. The setup and Wi-Fi update helpers also add their includes to existing Compose configuration. On the PowerGlove Vision Controller, check the published port with:

```
SH
docker port powerglove-vision-profile-relay-1 55356/udp
```

Expect a host binding for port `55356`. If it is missing, update the application and rerun PowerGlove Vision Controller setup. On RetroPie, compare the game's actual filename, including `.nes`, `.zip`, or `.7z`, with `/etc/powerglove/games.json`. These are separate exact entries. Updating the template does not overwrite an installed registry; add missing names while preserving your custom mappings.

Players, Academy progress, and hand-setup backups

The player selectors on Dashboard, Glove Academy, and Setup select the active player on this Controller, across browsers and game profiles. Up to twelve players with names of 1-32 characters can be stored. **Add player** copies current sensitivity, starts fresh lesson progress, and selects the new player. Rename/delete controls are under **Setup -> Players -> Players and hand-setup backups**; at least one player is retained.

Completed lessons, the current lesson, and Glove Master persist across refreshes and restarts. Skips do not count; **Start again** resets the active player's progress. Other tabs notice switches/resets, and stale writes cannot undo a reset or update another player. Saving errors pause lesson recognition until saved state is available again.

Switching players, adding/deleting the active player, and restoring settings pause controller output. Each player keeps a separate saved calibration. Selecting a player immediately loads their sensitivity, progress, and saved center. Use **Center hand** after moving the camera or changing playing position. A new player has no saved center and needs centering once. Saved centers apply through the durable restore path; output stays paused until you explicitly start it. Finish tuning and turn **Tune gestures** off before changing players or restoring settings.

Where player settings and backup files live

All players are saved automatically on the PowerGlove Vision Controller in `/home/arduino/ArduinoApps/powerglove-vision/data/gesture-tuning.json`. This single store holds each player's sensitivity, calibration, and Academy progress; there is no separate automatic file for each player. The active working calibration is also mirrored in `data/calibration.json` in the same application directory. Use Glove Academy to manage these settings.

Choose each player in turn and select **Back up hand setup** to download a separate file named for that player, such as `iain-powerglove-hand-setup.json`. Your browser saves it on the computer, phone, or tablet you are using, usually in **Downloads** or the folder you choose. To restore, select the player you want to update, choose **Restore hand setup**, and pick that player's saved file from your device. Review it before confirming; restore updates the selected player, rather than adding a new one.

The downloaded file is a portable copy, separate from the Controller's live player store. Exporting does not create a second backup file on RetroPie or the Controller. The browser controls the download folder and may add a number to repeated filenames. Keep a clearly named copy for every player you want to recover.

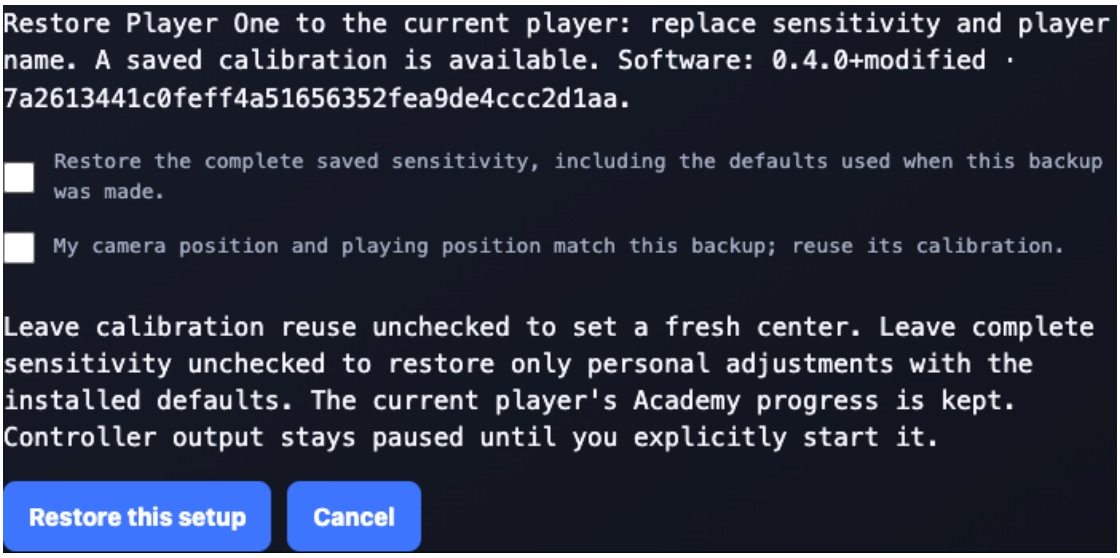
Backup contents and restore choices

Back up hand setup downloads `<player>-powerglove-hand-setup.json` with format `powerglove-hand-setup` and version `2`. Fields are `name`, personal `thresholds`, `calibration`, `effective_thresholds`, and `source` (`version`, `commit`). Empty personal thresholds mean no personal overrides. Effective thresholds contain all thirteen activation/release pairs, including the supplied defaults in use. They let a later restore retain those sensitivity values when defaults change. Game mappings, recognition algorithms, and all other software behavior are not frozen by a hand backup.

Calibration contains version `2` and `neutral` values: `palm_x`, `palm_y`, `palm_scale`, `roll`, `noise_x`, and `noise_y`. It comes from this player's saved reference, including while a selected player's saved center is being applied. It is `null` if this player has no saved reference. The app does not assume that a stored center still matches the present physical setup.

Restore hand setup opens a review before any changes. It replaces the active player's name and sensitivity while keeping Academy progress. Check **Restore the complete saved sensitivity** to use `effective_thresholds`; leave it unchecked to restore personal adjustments with the installed defaults. Independently, check **My camera position and playing position match this backup** to reuse calibration. Otherwise set a fresh center. Controls stay paused until Start.

Version 2 is the first supported portable backup format. Existing version-2 files without `effective_thresholds` or `source` still restore their personal adjustments and calibration. Version-1 `powerglove-hand-settings` exports are rejected without changing anything. Cancel closes the review without changes.



Review before restoring a complete hand setup

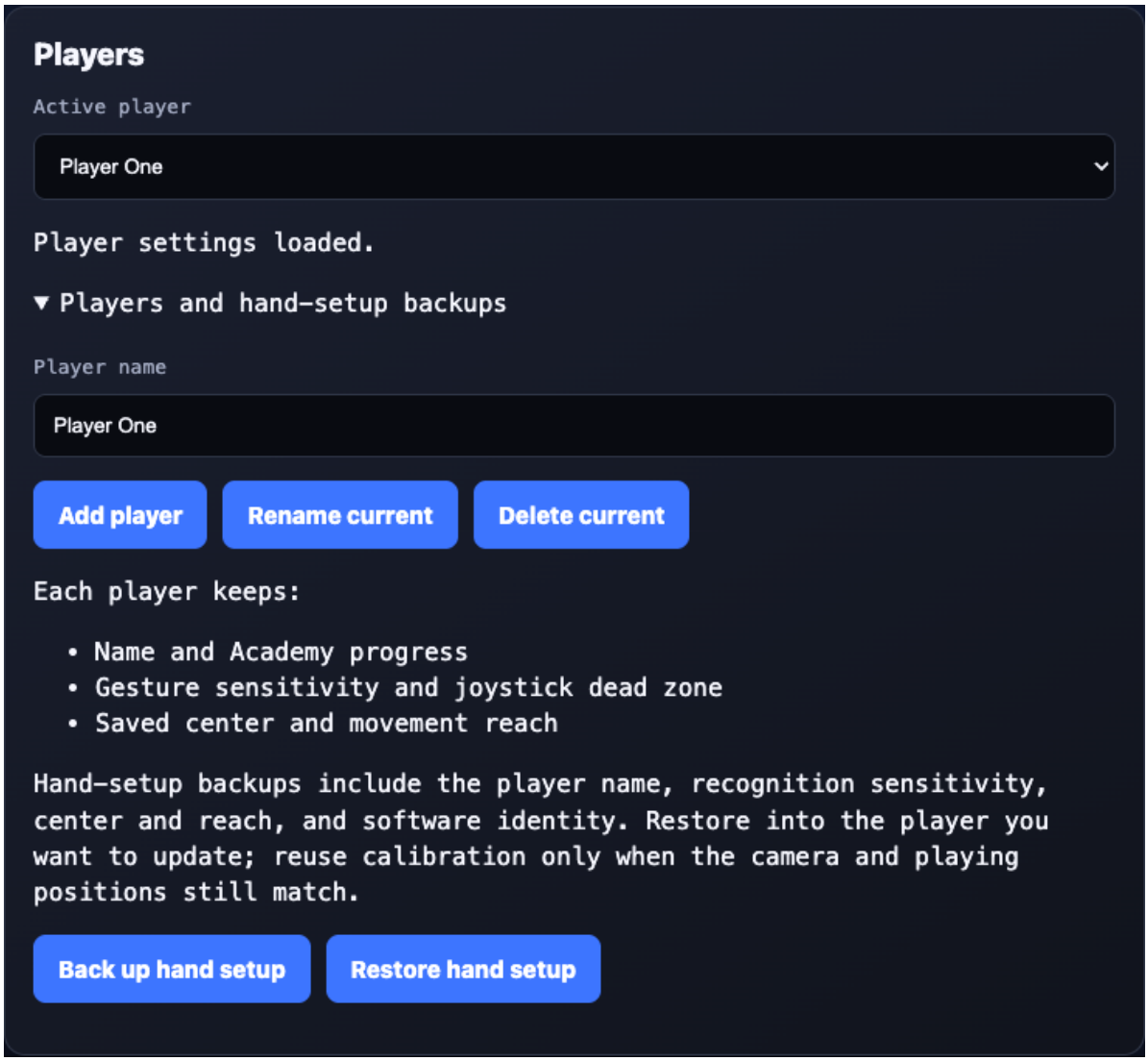
Backups exclude pairing tokens, Wi-Fi credentials, device addresses, images, landmarks, and Academy progress. Unknown fields, non-finite/out-of-range values, device configuration files, and files larger than 8 KB are rejected. The API requires boolean `reuse_calibration: true` for backup calibration reuse and `use_effective_thresholds: true` for complete sensitivity restoration.

`data/gesture-tuning.json` version 4 stores `version`, `active`, `generation`, `players`, and nullable `calibration_restore`. Each player has `name`, `thresholds`, `progress` (`course`, `completed`, `lesson`), `needs_center`, and nullable `calibration`. Course version 1 uses sixteen zero-based lesson indices. Generations reject stale writes after switches/restores/resets. The active working reference is mirrored in `data/calibration.json`; individual references are kept in the player store. Migration associates an existing valid reference only with the currently centered player, not with every preset.

Confirmed reuse atomically stores a pending calibration while keeping output gated. The worker writes the active calibration, then clears the pending reference and centering gate. An interrupted restore resumes after restart; a failed write leaves output paused. Switching players cancels an unapplied reference. Export waits until a pending restore finishes.

Internal store versions 1-3 migrate without losing names, sensitivity, or progress. Before the first write, `data/gesture-tuning-vN-backup.json` retains the old store, where N is its version. This internal recovery migration is separate from the unsupported version-1 portable export format. Files use mode `0600` and survive upgrades. Older apps cannot read version 4; stop the app and restore the appropriate private store backup when deliberately rolling back.

`POST /api/players` supports `read`, `progress`, `reset_progress`, `create`, `select`, `rename`, `delete`, `export`, `restore`, and `reuse_calibration`. Non-read requests include `player` and `generation`. Saved-player reuse also requires `confirmed: true`. JSON bodies are limited to 8192 bytes and require `X-PowerGlove-Action: players` and the same origin checks as tuning. Names and progress are available on the trusted LAN; presets are not login accounts.



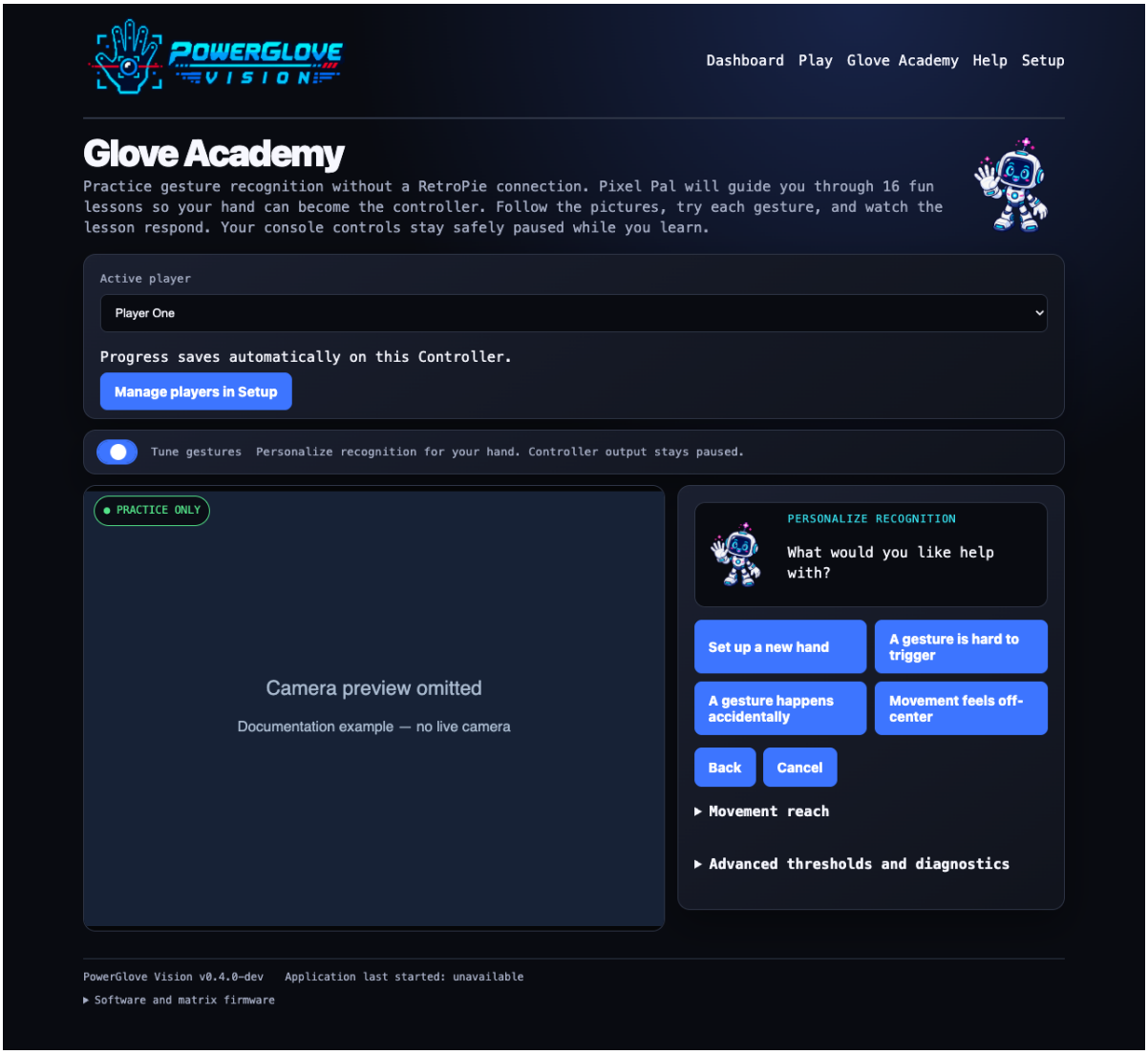
Player selection and portable hand-setup backups

Tune gesture sensitivity

Use **Glove Academy -> Tune gestures** to personalize recognition. You do not need to edit `config/profiles.json`; it is the release-owned shared baseline. Updates back up and replace it. Personal adjustments belong in `data/gesture-tuning.json`, which remains untouched.

1. Choose **Set up a new hand**, **A gesture is hard to trigger**, **A gesture happens accidentally**, or **Movement feels off-center**.
2. Choose the gesture when asked. Off-center movement instead shows the saved center and an explicit **Center hand** action.
3. Keep the complete hand visible with valid palm geometry for one second. Select **I'm ready** and wait through the two-second countdown. The displayed MediaPipe score is handedness certainty, not a position-quality requirement.
4. Follow the three recordings. Ordinary poses and movement steps last two seconds. Glove Zap and Pull Back use a six-second middle step containing three motions and returns.

5. Analyze the recording and try the temporary preview twice. Return to neutral after each use and remain neutral for three seconds.
6. Save when the guided test passes. Only selected components are merged into the active player's hand settings.



Tune mode with Pixel Pal guiding the personalization choices

The reference screenshot deliberately excludes the live camera area. Pixel Pal presents one instruction and primary action at a time. **Movement reach** is a separate collapsed section for the four per-player spans and saves only those values. The numerical table and diagnostic capture are collapsed under **Advanced thresholds and diagnostics**. The matrix shows a scanning **T** while tuning and a matching scanning **L** in ordinary practice.

Activation is the point where a gesture begins; release is the lower point where it stops. Separate values prevent rapid on/off flickering. Gameplay movement mappings use these same held states, including wrist steering, push, pull-back, and braking; game-specific button assignments and pulses still apply. Directions and fingers can be adjusted independently. Compound gestures share component thresholds, so changing a finger also affects other gestures that use it. Suggested menu-pose adjustments tune the closed fingers; already extended fingers retain their existing settings from hand setup or existing personal/default values. Button assignments and menu hold timing remain unchanged.

Hand setup learns open and curled thresholds for all five fingers. Individual tuning can be used without setup; it only learns new thresholds for fingers observed both open and curled. Fingers extended throughout retain hand-setup thresholds or

existing settings. Feedback uses the same V-sign and thumbs-up checks as recognition. Hand setup reset restores all five finger components; individual reset restores only the selected components. Difficult gestures place activation 55% into the measured rest-to-action gap; accidental gestures use 75% activation and 40% release. Standard setup retains 65% activation and 30% release. Every path rejects a gap below 0.08.

Only the active player's adjusted components override all game profiles. Untuned components retain the shared supplied values. Personal adjustments are saved atomically in `data/gesture-tuning.json` and survive application restarts and normal updates. Normal personalization saves no images or recordings. Existing version-1 files migrate as described above; this legacy example remains readable:

```
JSON
{
  "version": 1,
  "thresholds": {
    "index": {"on": 0.6, "off": 0.4}
  }
}
```

Each pair must contain finite numbers with `0 <= off < on`. Finger and pull activation cannot exceed `1`; wrist rotation cannot exceed `2`; movement and push cannot exceed `4`. These are normalized measurements, not distances in centimetres.

Tuning pauses controller delivery. A game launch may update the selected game but cannot interrupt tuning or send game input. Leaving Tune discards its preview; a disconnected browser's session expires after six seconds. Return to Dashboard and explicitly start controller delivery when ready to play. **Recalibrate neutral** changes the resting reference separately and invalidates any current recordings.

Private Academy diagnostic capture

The Advanced diagnostic is separate from personalization. Eight user-paced cues exercise neutral, directions, A/B, menu poses, rolls, depth motion, Menu Guard, and tracking recovery using the deployed **MediaPipe Hands** backend. The PowerGlove Vision Controller records a temporary local AVI only while a cue is active. Completion produces an aggregate JSON report containing detection continuity, confidence, latency, hand brightness, and recognized state names. It contains no frames or per-frame landmarks. The AVI is deleted immediately after analysis or cancellation; an abandoned capture is deleted after 30 minutes. No network upload occurs.

Supplied shared recognition defaults

The following fields describe the shipped `config/profiles.json`. They remain useful for understanding the defaults; personal tuning is managed through Glove Academy.

Threshold fields

Field	What it measures	Effect of lowering the value
`move_on`	Palm displacement from center, normalized by palm size	Movement activates sooner
`move_off`	Palm displacement at which active movement releases	Movement stays active farther back toward center
`coordinate_edge_margin`	Camera margin excluded from native X/Y travel	Native travel reaches its edge closer to the camera boundary
`coordinate_smoothing_min`	Minimum weight assigned to the newest native coordinate	Small native movements respond more immediately but may show more jitter
`coordinate_smoothing_max`	Maximum newest-coordinate weight during deliberate travel	Large native movements catch up less quickly
`coordinate_motion_boost`	How quickly movement raises the coordinate weight toward its maximum	Native travel receives less immediate acceleration
`motion_noise_multiplier`	Multiplier applied to measured per-player resting jitter for native X/Y	Less resting motion is suppressed
`motion_noise_floor`	Minimum native X/Y noise floor in normalized camera units	Smaller camera fluctuations can move the native hand
`motion_noise_exit_ratio`	Hysteresis boundary for leaving the native X/Y noise floor	Intentional movement exits the resting region sooner
`motion_slow_follow`	Newest-coordinate weight immediately beyond the native noise floor at the configured follow-reference interval	Slow native movement is more damped
`motion_full_speed`	Calibrated reach spans per second at which native X/Y becomes one-to-one	Full-speed following begins at a slower hand speed

Field	What it measures	Effect of lowering the value
`motion_follow_reference_ms`	Interval at which `motion_slow_follow` is interpreted; 100 ms matches the proven Controller's MediaPipe cadence	The same weight is applied over a shorter interval, increasing damping at a given observation cadence
`curl_on`	Normalized finger curl, where `0` is straight and `1` is tightly curled	Curl actions activate with less bend
`curl_off`	Curl amount at which an active curl releases	Curl stays active until the finger is straighter
`roll_on`	Wrist rotation from the centred angle	Roll actions activate with less rotation
`roll_off`	Rotation at which active roll releases	Roll stays active closer to neutral
`push_on`	Relative increase in apparent hand size from center	Push actions activate with less forward movement
`push_off`	Depth change at which an active push releases	Push stays active closer to the centred depth
`depth_confirm_frames`	Consecutive beyond-threshold observations required for Glove Zap or Pull Back	Fewer observations accept shorter changes but reduce spike rejection
`depth_motion_window_ms`	Window in which the required depth travel must occur	A longer interval accepts slower depth motion
`depth_motion_delta`	Minimum normalized palm-scale travel toward or away from the camera	Smaller apparent-size changes can qualify as depth actions
`pulse_hz`	Repetition rate for profiles that pulse an action	Repeated actions become slower
`loss_release_ms`	Tracking-loss delay before all controls release	Controls release sooner after the hand disappears

For each gesture, keep the `_off` value lower than its `_on` value. The gap is hysteresis: it prevents a value near the activation point from rapidly turning on and off. A very large gap can make the control feel sticky.

The supplied shared recognition defaults are:

```JSON
{
  "move_on": 0.28,
  "move_off": 0.14,
  "coordinate_edge_margin": 0.08,
  "coordinate_smoothing_min": 0.70,
  "coordinate_smoothing_max": 1.00,
  "coordinate_motion_boost": 4.00,
  "motion_noise_multiplier": 1.25,
  "motion_noise_floor": 0.003,
  "motion_noise_exit_ratio": 1.50,
  "motion_slow_follow": 0.70,
  "motion_full_speed": 1.50,
  "motion_follow_reference_ms": 100.0,
  "curl_on": 0.50,
  "curl_off": 0.35,
  "roll_on": 0.58,
  "roll_off": 0.40,
  "push_on": 0.34,
  "push_off": 0.18,
  "depth_confirm_frames": 2,
  "depth_motion_window_ms": 250,
  "depth_motion_delta": 0.10,
  "pulse_hz": 7.0,
  "loss_release_ms": 120
}
```

The `recognition` object applies to every game profile. During neutral calibration, the worker records ordinary X/Y jitter and raises only the relevant movement thresholds when necessary; it never lowers the `0.28` activation or `0.14` release baselines. Programs A-I and dedicated game profiles only decide how those shared recognition states map to controller output.

Native stabilization treats a saturated `1.0` calibration jitter measurement as unusable. It keeps that calibration's center, scale, and wrist values but uses the small fixed native noise floor, avoiding a large dead zone followed by a jump. A normal measured jitter value continues to raise the native noise floor.

When adjusting numeric values in Tune, change one pair at a time in steps of approximately `0.02` to `0.05`, then test from the same camera position. Useful adjustments include:

- Recalibrate first if directional movement requires too much travel or moves at rest.
- Keep `move_off` below `move_on` so a direction releases promptly near center.
- Raise an `_on` value when an action triggers unintentionally.
- Increase `pulse_hz` when a repeating action is too slow.
- Keep `loss_release_ms` short enough to release safely but long enough to tolerate a few missed camera frames.

Saved personal tuning applies without reopening the camera and across every profile. The saved neutral calibration is reused; recalibrate only if your physical setup has changed or your resting hand position produces unwanted movement. Gesture-to-button assignments are implemented by each profile in the application; threshold changes adjust sensitivity but do not remap buttons.

Signed controller transport and upgrades

Controller traffic uses `powerglove-vision/2` on UDP `55355`. Domain-separated HMAC-SHA256 covers each entire canonical JSON message, including its kind, session, request/challenge, and input fields. The shared token stays in the existing private configuration files. Messages are authenticated, not encrypted; the trusted-LAN requirements still apply.

A signed hello obtains a receiver-issued challenge before input can be accepted. Session identifiers, request identifiers, and challenges are random 128-bit hex values. The first valid state activates the challenge and retires all earlier challenges; each later state needs an increasing sequence. Receiver restarts require a fresh challenge and do not depend on synchronized clocks. Pending handshakes are limited to eight and expire after three seconds. Datagrams remain bounded to 4096 bytes; duplicate JSON fields, malformed controls, wrong signatures, and retired or replayed input are rejected before uinput/native-state publication.

The sender never queues input during negotiation. It retries hello after 250 milliseconds until established, then every second so a restarted receiver can issue a fresh challenge. Replies return from UDP `55355` to the sender's ephemeral UDP socket. On Linux the receiver uses IP_PKTINFO to reply from the address/interface contacted, preserving container NAT routing when Ethernet and Wi-Fi share a subnet. As an additional compatibility measure, the sender requires the configured source port plus a valid HMAC, session, and fresh request identifier rather than an identical source IP. The sender processes at most eight replies per update without waiting. A successful UDP send is not an acknowledgement of emulator consumption. Handshake/rejected traffic cannot postpone the 250-millisecond release deadline.

Upgrade both computers together

1. Select **Stop controller** and back up both installations and private settings.
2. Update RetroPie and the PowerGlove Vision Controller from the same development commit or compatible release. The default new receiver rejects old input, and the new sender does not downgrade to version 1; mixed versions will pause controller delivery.
3. Restart both applications, confirm matching software identities, then select **Start controller**. Verify neutral/release behavior and actual game input. Profile changes also establish a fresh controller session.
4. If you must roll back, stop controls and restore both matching application versions. Preserve device settings, calibration/player files, and the paired token; do not restore a mismatched sender/receiver combination.

For a staged upgrade only, the new receiver has `--allow-legacy-controller`. An administrator can temporarily add it to the receiver invocation while the older Controller is being replaced. It is off by default, still exposes the shared token in legacy traffic, and closes for the rest of that receiver process after the first valid version-2 state. Remove the flag after upgrading; a receiver restart would otherwise reopen legacy admission. This compatibility mode does not provide version-2 replay protection. Re-pair after migration if a token may have been observed in old traffic: signing cannot revoke a previously exposed key.

Existing pairing credentials and native emulator files need no format migration. Installation packages include the new protocol and web modules; neither a firmware flash nor a core rebuild is required by this transport change. The four-pixel Off display requires its matching matrix firmware. Once installed, extending the fourth pixel to Ethernet requires only the updated Controller app and host sampler.

RetroPie receiver and virtual controller

The receiver verifies authenticated UDP packets and creates a Linux `uinput` gamepad named `PowerGlove Vision`. Its installed service is:

```
/etc/systemd/system/powerglove-receiver.service
```

The supplied service listens on all local interfaces at UDP port `55355`, reads `/etc/powerglove/token`, and releases held controls when a socket receive times out after 250 milliseconds. The receiver measures its release deadline from the last valid packet; rejected traffic cannot postpone release. Both the virtual gamepad and native state are neutralized on timeout. If you change the controller port in PowerGlove Vision Controller Setup, add the same `--port` value to the service's `ExecStart`, then reload and restart:

```
SH
sudo systemctl daemon-reload
sudo systemctl restart powerglove-receiver.service
```

The companion `powerglove-receiver.timer` starts the receiver 45 seconds after boot. This lets EmulationStation finish its initial controller scan first. Keep the timer enabled and the service itself disabled for boot activation. Starting the receiver too early can cause frontend pauses and conflicts with other USB devices, such as a BitPixel display.

```
SH
systemctl is-enabled powerglove-receiver.timer
systemctl is-enabled powerglove-receiver.service
sudo systemctl status powerglove-receiver.service
sudo journalctl -u powerglove-receiver.service -n 100 --no-pager
```

The virtual gamepad appears only after the first authenticated controller packet. On the PowerGlove Vision Controller, select **Start controller** and show a centred hand before deciding that the device is missing.

RetroArch autoconfiguration

Install the supplied mapping at:

```
/opt/retropie/configs/all/retroarch/autoconfig/PowerGlove Vision.cfg
```

It matches the virtual device name plus vendor and product IDs `1:1`, uses the `udev` input driver, and maps D-pad directions, A, B, Start, Select, and four axes to a standard RetroPad. It does not configure an I-PAC, 8BitDo controller, or any other physical controller.

If RetroArch has a hand-written override for this device, remove or reconcile that override before diagnosing the supplied autoconfiguration.

PowerGlove Vision Controller privileged host helpers

The standard installation includes the host shutdown helper so Dashboard and Setup can halt Linux cleanly. It consists of two systemd units and one boot-time readiness rule:

Repository file	Installed path	Purpose
`uno-q/powerglove-system-shutdown.path`	`/etc/systemd/system/powerglove-system-shutdown.path`	Watches for one fixed shutdown request in the application's private data directory
`uno-q/powerglove-system-shutdown.service`	`/etc/systemd/system/powerglove-system-shutdown.service`	Removes that request and asks systemd to halt Linux cleanly
`uno-q/powerglove-system-shutdown.conf`	`/etc/tmpfiles.d/powerglove-system-shutdown.conf`	Recreates the unprivileged readiness marker at boot or after application replacement

The standard installer adds `powerglove-camera-recovery.path`, its fixed-purpose service, `/usr/local/libexec/powerglove-camera-recovery`, and a tmpfiles rule. Installation does not require a camera. If exactly one UVC camera is connected, the helper records it and its nearest external parent hub immediately; otherwise enrollment is deferred until vision first sees the camera successfully. The root-owned `/etc/powerglove-camera-recovery.json` allowlist stores the observed camera identity plus the hub identity and physical USB path. A later healthy sighting updates the association automatically if the camera has moved.

During an outage the unprivileged application can request only the helper's fixed operation. It validates the stored hub path and identity before resetting that hub, waits for one UVC camera to enumerate, updates the allowlist, and sets the camera and hub power policies to `on`. It never guesses among hubs. A root-owned 60-second cooldown and the application's one-request-per-outage rule prevent reset loops. Resetting the hub can briefly interrupt USB Ethernet and any other devices attached to it. Before the first successful camera sighting, there is deliberately no reset target; reconnect or power-cycle the camera once to let automatic enrollment complete.

Install all host helpers with `scripts/install-uno-q-shutdown-helper.sh`, or install/repair only camera recovery with `scripts/install-uno-q-camera-recovery-helper.sh`. The standard installer also creates the private `data/.shutdown-enabled` marker that allows the web UI to offer the action and `data/.camera-recovery-enabled` for camera recovery. The tmpfiles rules restore both markers at boot. The application cannot use this mechanism to execute an arbitrary privileged command; it can only create the two fixed request files.

Do not change the request path in only one component. The web application, path unit, service, and marker must continue to agree. After installation, confirm that `powerglove-system-shutdown.path` and `powerglove-camera-recovery.path` are enabled and active.

Network ports and trust boundary

Keep PowerGlove Vision on a trusted home or cabinet LAN. Do not forward these ports through a router or expose them directly to the Internet.

Port	Direction	Purpose
UDP `55355`	PowerGlove Vision Controller to RetroPie	Authenticated live controller state

Port	Direction	Purpose
UDP `55356`	RetroPie to PowerGlove Vision Controller	Authenticated game-profile requests and acknowledgements
TCP `8088`	Browser to PowerGlove Vision Controller	Dashboard, Play, Help, Glove Academy, and ordinary Setup UI, including Games
TCP `8443`	Browser to PowerGlove Vision Controller	TLS Setup and pairing workflow
TCP `55358`	PowerGlove Vision Controller to RetroPie	Paired game registry reads, saves, and restoration
TCP `55357`	PowerGlove Vision Controller to RetroPie	Temporary one-time-code pairing helper

The two UDP ports serve different purposes despite their similar numbers. The `port` in PowerGlove Vision Controller `device.json` is normally `55355`; the `port` in RetroPie `launcher.json` is normally `55356`.

The ordinary dashboard is reachable by other devices on the LAN. Pairing and shutdown require additional protections, but the project assumes that the LAN itself is trusted. Review SECURITY.md before using a shared, guest, school, or public network.

PowerGlove Vision Controller matrix artwork

The built-in display is a 13-by-8 monochrome blue LED matrix with eight brightness levels. Treat it as a very small dot-matrix display in the spirit of a pinball DMD or monochrome BitPixel display, rather than as a miniature screen. The sketch encodes status, pairing, profile identifiers, and the gestures-idle Power Glove attract sequence. Glove Academy and Tune share a dim letter, bright scan line, and trailing glow: eight frames at 160 milliseconds each (a 1.28-second cycle).

Matrix artwork should use broad motion, recognizable silhouettes, and strong separation between the subject and its brightest effect. A moving spark needs a dim body underneath it, a medium halo, and a full-bright core. Pulses should emphasize an outline instead of illuminating every pixel at maximum brightness, which erases the shape on the physical display. Transitional objects need to cross several columns and remain visible for more than one frame; isolated one-pixel changes are easily lost to persistence and viewing angle.

Always judge animation timing and grayscale on the physical PowerGlove Vision Controller. A source grid or browser mock-up is useful for finding malformed frames, but it cannot reproduce LED bloom, exposure, or perceived persistence. A short video covering several complete loops is the preferred review artifact for later refinements.

Files most users should not edit

File or directory	Purpose
`app.yaml`	Arduino App Lab application metadata and exposed ports
`sketch/sketch.yaml`	PowerGlove Vision Controller sketch platform and pinned Arduino library dependencies
`pyproject.toml`	Python package metadata, supported interpreter range, and optional dependencies
`python/worker-wheels/`	Platform-specific MediaPipe worker dependency supplied by the App Lab installation ZIP
`data/models/hand_landmarker.task`	Checksum-verified cached model, installed from the bundle when vision is first activated
`data/uv-cache/` and `data/uv-python/`	Generated private worker runtime and package cache
`.cache/app-compose.yaml`	App Lab generated container configuration
`data/.shutdown-enabled`	Readiness marker installed by the fixed-purpose shutdown helper included in standard setup
`data/.camera-recovery-enabled`	Readiness marker for the fixed-purpose host USB-camera recovery helper
`data/camera-recovery-request`	Ephemeral one-shot request consumed by the root-owned camera recovery service
`output/pdf/`	Generated PDF editions; public editions are served by Help, while the cabinet quick reference remains private

Changing manifests can prevent App Lab from starting the application. Generated files and private runtime caches should not be committed or added to an App Lab installation ZIP. The installation ZIP is built as:

```
output/app-lab/PowerGlove-Vision-Uno-Q.zip
```

The installation ZIP includes the verified model at `models/hand_landmarker.task`, its Apache 2.0 license, and third-party notices. It excludes private `data/`, caches, tests, Git metadata, and the cabinet-specific quick-reference PDF. It includes only the allowlisted public PDF editions used by Help. When an active profile first needs vision, the application installs the bundled Google Hand Landmarker model into its private cache and verifies its SHA-256 checksum. A download is attempted only if the bundle is absent. The model stays unopened while **Gestures off** is selected.

Automated quality and package verification

Maintainers use `.github/workflows/quality.yml` for pull requests, pushes to `main` and `dev`, and manual runs. The workflow tests the project on supported Python versions, checks Python and shell syntax, validates JSON, audits source and documentation, rebuilds and inspects the PDF set, and verifies the App Lab installation ZIP. The workflow has read-only repository permissions and uses no deployment credentials.

Ordinary installers do not need to edit this workflow. If you fork the project, keep live PowerGlove Vision Controller credentials and cabinet deployment out of hosted CI; deployment should remain an explicit action from a trusted machine.

Back up and update safely

Back up custom configuration before replacing an installation:

Item	Why it matters
PowerGlove Vision Controller `data/calibration.json`	Preserves your neutral hand position, size, and wrist angle; recalibrate if the physical setup changes
PowerGlove Vision Controller `data/device.json`	Contains device settings and the private token
PowerGlove Vision Controller `config/profiles.json`	Release-owned shared defaults; updates back up and replace this file
PowerGlove Vision Controller `data/gesture-tuning.json`	Contains saved personal gesture sensitivity values and is preserved
RetroPie `/etc/powerglove/games.json`	Contains local ROM mappings
RetroPie `/etc/powerglove/launcher.json`	Contains local host and path settings
RetroPie `/etc/powerglove/token`	Contains the matching private token
RetroArch `PowerGlove Vision.cfg`	Contains any deliberate local mapping changes

Store token-bearing backups privately with restricted permissions. The supplied Wi-Fi deployment script preserves the PowerGlove Vision Controller `data/` directory. The App Lab installation ZIP never contains your token or private model cache; it includes the unmodified public model.

After an update, confirm that the active files under `/etc/powerglove/` still contain your local hostnames and ROM names. Updating repository templates does not automatically migrate active configuration.

Troubleshooting by symptom

Symptom	Configuration checks
Dashboard works but no virtual controller appears	Start the controller, show a centred hand, verify the receiver service and shared token, then check UDP `55355`.
Controller appears but a game uses the wrong gestures	Confirm the system is `nes` or `famicom` and the exact ROM basename exists in `/etc/powerglove/games.json`.
Game launches slowly while PowerGlove Vision Controller is offline	Confirm `timeout` remains near `0.4`; the hook retries but must never block game launch indefinitely.
Profile command is not acknowledged	Check the PowerGlove Vision Controller name, UDP `55356`, pairing token, and the PowerGlove Vision Controller application status.
Gestures off shows a blinking X	Update PowerGlove Vision; Gestures off should show the glove attract animation and must not open the camera.
Camera disappears after reboot	Check `lsusb` and `/dev/v4l/by-id/`, reconnect the camera or hub if absent, and keep Camera set to **Automatic** unless selecting a specific listed device. See startup diagnostics.
First activation is slow	Allow background preloading to finish and inspect the startup stage logs before attributing the delay to the camera.
Movement triggers too late	Recalibrate neutral first and verify the hand is steady; all profiles share the responsive movement thresholds.
Direction remains stuck	Recalibrate neutral, verify return toward center and tracking-loss release, then review the shared `move_off` value.
Pairing suddenly fails after a Setup change	A rotated token invalidates the old pairing; run the pairing flow again.
EmulationStation pauses or another USB device behaves unexpectedly at boot	Verify receiver startup is controlled by the 45-second timer and the service is not independently enabled at boot.

Configuration file catalog

Repository file	Active or installed copy	Used by
`config/device.example.json`	PowerGlove Vision Controller application `data/device.json`	Vision supervisor and Setup UI
`config/profiles.json`	PowerGlove Vision Controller application `config/profiles.json`	Gesture engine
`config/games.json`	RetroPie `/etc/powerglove/games.json`	Launch hook and profile selector
`config/launcher.example.json`	RetroPie `/etc/powerglove/launcher.json`	RetroPie launch and exit hooks
`retropie/retroarch/PowerGlove Vision.cfg`	RetroArch autoconfig directory	RetroArch input system
`retropie/powerglove-receiver.service`	`/etc/systemd/system/`	Privileged virtual-controller receiver
`retropie/powerglove-games.service`	`/etc/systemd/system/`	Paired Games editor service on RetroPie
`data/gesture-tuning.json` (runtime only)	PowerGlove Vision Controller application `data/gesture-tuning.json`	Player sensitivity, per-player calibration, and Academy progress
`retropie/powerglove-receiver.timer`	`/etc/systemd/system/`	Delayed boot activation
`uno-q/powerglove-system-shutdown.path`	`/etc/systemd/system/`	Fixed shutdown request watcher
`uno-q/powerglove-system-shutdown.service`	`/etc/systemd/system/`	Fixed clean-shutdown action
`uno-q/powerglove-system-shutdown.conf`	`/etc/tmpfiles.d/`	Boot-time shutdown readiness marker
`uno-q/powerglove-camera-recovery.path`	`/etc/systemd/system/`	Watches the fixed camera-recovery request
`uno-q/powerglove-camera-recovery.service`	`/etc/systemd/system/`	Runs the bounded camera recovery action
`uno-q/powerglove-camera-recovery.py`	`/usr/local/libexec/powerglove-camera-recovery`	Enrolls one UVC camera and resets only its last observed, identity-checked parent hub

Repository file	Active or installed copy	Used by
`uno-q/powerglove-camera-recovery.conf`	`/etc/tmpfiles.d/`	Boot-time camera-recovery readiness marker
Runtime camera allowlist	`/etc/powerglove-camera-recovery.json`	Root-owned camera identity and last successfully observed hub path and identity
`.github/workflows/quality.yml`	GitHub Actions	Automated tests and release verification
`app.yaml`	PowerGlove Vision Controller application root	App Lab
`sketch/sketch.yaml`	PowerGlove Vision Controller application sketch directory	Arduino build system
`pyproject.toml`	Repository or deployed application root	Python packaging and tests

When a change goes wrong, restore the last known-good active file rather than copying every repository template over the installation. Validate JSON, restart only the affected service or application, and test packet delivery before changing controller mappings.

Local hostname resolution inside App Lab

UNO host setup limits Avahi to detected physical network interfaces, excluding Docker bridges and virtual Ethernet devices. On the test cabinet this prevented the conflict rename to `ArduIain-2.local` seen during app restarts, and restored automatic game-profile heartbeat delivery. The helper `sudo python3 scripts/configure-uno-q-avahi.py` preserves the original config as `/etc/avahi/avahi-daemon.conf.powerglove-backup`; restart `avahi-daemon` after running it manually. Rerun it if replacing a USB Ethernet adapter changes the interface name. It does not change your hostname, addresses, or network links.

If the console name fails, use **Check console address** in Connection, then follow hostname troubleshooting. A router-reserved IPv4 address is a fallback, not a setup requirement.

Both machine installers install `avahi-daemon` and `libnss-mdns` and enable Avahi at boot. The PowerGlove Vision Controller host dependency supports native hostname lookups; it does not replace the app-owned resolver used inside the container. Setup check mode verifies the dependency and Avahi service, and the PowerGlove Vision Controller check tests the configured destination inside the app.

The app-owned `local:avahi_resolver` brick survives App Lab container regeneration. It mounts `/run/avahi-daemon` read-only in a separate unprivileged service and exposes only IPv4 `.local` queries through `data/.avahi-resolver.sock`. It has no network interface or published port. The app prefers this private socket; the direct host socket remains a compatibility fallback. Both sockets are runtime files, not configuration to back up or distribute.

Gameplay and pairing use this resolver. Controller-state sends use a background address refresher: one lookup at a time, refreshed every five seconds and retried after two seconds on failure. The send path only reads its cached address. It drops that frame when no address is ready; it does not queue states for later. A last successful address expires after ten seconds if refresh stops succeeding. Literal IPv4 addresses need no background lookup. Pairing and administrative

requests still resolve synchronously outside the movement loop. Resolver answers expire after five seconds, so DHCP changes do not require editing an address. Ordinary DNS names use the system resolver. Generic container `getent` is not the app's mDNS test; use Connection's hostname test or the setup command's check mode.

Independent Networking status

The fourth Setup marker and Off-mode pixel report the Controller's physical Wi-Fi or Ethernet link, including Ethernet through a USB dock. The host sampler reads carrier state under `/sys/class/net`; physical Ethernet must have a device entry and Ethernet type. It ignores loopback, Docker bridges, and virtual veth interfaces. Any connected relevant interface makes the result connected. All observed relevant links down means disconnected; missing or unreadable evidence means unavailable unless another relevant link is confirmed up.

This is link health, not an IP-address, Internet, routing, or game-delivery test. The other three markers show app, console-service, and authenticated-response status. Green means confirmed, red disconnected/not confirmed, grey unknown. Tracking, controller output, and saved console appear below the markers.

The existing `powerglove-wifi-status.timer` runs every five seconds and invokes `powerglove-wifi-status.service`. The unprivileged helper publishes `data/wifi-status.json` with version 1, `observed_at`, wireless-only `state`, and aggregate `networking`. These literal filenames and the old wireless field are retained for upgrade compatibility. No credentials, interface names, SSIDs, or addresses are recorded. Reports expire after fifteen seconds; malformed, missing, or future-dated data is unavailable. An older report without `networking` can confirm connected Wi-Fi, but disconnected Wi-Fi cannot rule out Ethernet and therefore yields unknown aggregate status.

The read-only `/api/connection-status` endpoint shares the matrix cache. Visible Setup pages poll without overlap every five seconds with a 3.5-second request timeout. Console checks run in one background thread at most every ten seconds and expire after thirty; changes to destination or token invalidate them. Setup can request checks in any attract mode; idle Off also refreshes them.

Normal installation and `scripts/deploy-uno-q-wifi.sh` update the sampler. Repair it separately with `sudo python3 scripts/setup-machine.py uno-q --wifi-status-only`. The existing four-pixel firmware and RetroPie need no update for Ethernet indication; a Controller application and host-sampler upgrade is required. Game/profile artwork, T/L, startup, and pairing displays remain unchanged.

Known limitation: PowerGlove Vision Controller restarts after Shutdown

On the tested PowerGlove Vision Controller, a graceful `halt` still leads to an automatic restart, including when connected directly to a Mac without the powered hub. The helper requests halt correctly, but remaining stopped is not verified. Do not use loss of the website or a fixed delay as confirmation that power can safely be removed. See the installation guide for the investigation status and Arduino guidance.

Saved neutral-hand calibration

The worker saves its completed neutral reference in `data/calibration.json`. It includes palm position, apparent size, wrist angle, and normal X/Y positional jitter; it is not a personally trained gesture model. The jitter estimate can raise the shared movement thresholds above their baseline, but never makes them more sensitive. Glove Academy, gameplay, profile changes, camera reconnects, and worker restarts reuse this reference. **Center hand** explicitly replaces it after sampling completes; an interrupted calibration preserves the previous saved reference. Recalibrate after moving your camera or changing your seating or standing position.

On first use, or if the saved file is missing or invalid, the worker samples an initial reference automatically. Calibration requires 24 complete observations at 70% hand confidence or better. It averages palm center and apparent size, uses a circular mean for wrist angle, and records the 95th-percentile X/Y deviation as normal jitter. Hold a relaxed open hand still at the intended neutral point and distance. Repeating from the same physical setup should produce a close reference, not identical floating-point values, because camera landmarks vary from frame to frame.

The release-owned `config/profiles.json` contains the portable starting point: movement thresholds, coordinate range, stabilization, finger, roll, depth, pulse, and tracking-loss defaults. The installer's package validator requires that file and the recognition modules, and updates back up and replace the shared profile file. It deliberately excludes `data/calibration.json`; copying one person's palm coordinates and apparent scale to another camera would create a misleading neutral point. Existing neutral calibration and personal gesture tuning remain preserved during updates. A storage failure is reported as `calibration_save_error` in status; the reference remains usable in memory but will not survive a worker restart.

Command-line reference

Use this section to look up a flag without interrupting the installation steps. It covers every project command and script, plus the external-command options used in these guides. Defaults describe this source version, not a guarantee about a future release. For external tools' other options, use their own help or manual pages.

An **option** begins with `-` or `--`. A **positional argument** is a value supplied in a particular place, such as `retropie` after `setup-machine.py`. Replace example hostnames and file paths with your own. Square brackets in usage descriptions mean optional; do not type them. Options that take a value need both the option and its value, such as `--port 55355`.

Install or inspect a machine

Run `sudo python3 scripts/setup-machine.py MACHINE [OPTIONS]` from the project directory on the target Linux machine. This is the recommended installer for both PowerGlove Vision Controller and RetroPie. `--check` performs read-only checks; using `sudo` also lets those checks read protected token files.

Argument or flag	Default	Meaning
`MACHINE`	Required	`retropie` or `uno-q`; selects the machine to install or inspect.
`--peer HOST`	None	Required for a new RetroPie launcher configuration; supplies the PowerGlove Vision Controller hostname or IPv4 address. Existing launcher settings are preserved. On PowerGlove Vision Controller, it prints guidance but does not change the saved receiver address.
`--check`	Off	Checks the existing installation without installing, restarting, or changing it.
`--wifi-status-only`	Off	With `uno-q`, install/update only the unprivileged Wi-Fi status sampler. Cannot be combined with `--check`. Normal setup and Wi-Fi deployment include it automatically.
`-h`, `--help`	-	Prints usage and exits.

Exit codes are `0` for success, `1` for an installation/check failure, and `2` for outstanding user action. Argument errors also use argparse's exit code `2`. The current check always asks for human gameplay confirmation.

Run the RetroPie receiver

Use `/opt/powerglove/bin/powerglove-receiver` on RetroPie. Normal operation is managed by its systemd service. A manual receiver cannot share the same port with that service: stop the service before a manual diagnostic run, then restart it afterward. Use `--token-file` rather than placing a token in shell history.

Flag	Default	Meaning
`--listen ADDRESS`	`127.0.0.1`	Local address to bind. `0.0.0.0` accepts packets on all local IPv4 interfaces, as the installed service requires.
`--port NUMBER`	`55355`	UDP port for controller packets; must match PowerGlove Vision Controller settings.
`--token VALUE`	None	Supplies the shared token directly. Use only as an advanced alternative; the value can appear in process arguments.
`--token-file PATH`	None	Reads the shared token from a protected file. Supply exactly one of this flag and `--token`. The token must contain at least 16 characters.
`--allow-legacy-controller`	Off	Temporary version-1 receiver compatibility during a staged upgrade; closes after the first signed state until process restart. Remove after upgrading.
`--timeout-ms NUMBER`	`250`	Socket receive timeout in milliseconds; a timeout releases held controls. Use a positive value.
`--native-state PATH`	`/run/powerglove/native-state`	Versioned latest-sample record for the optional custom Nestopia research core. Failure to create it does not disable FCEUmm/uinput.
`--dry-run`	Off	Prints received controls instead of creating a virtual input device.
`-h`, `--help`	-	Prints usage and exits.

Example: inspect packets without sending input to Linux. Run each command on RetroPie and press Ctrl+C to end the diagnostic receiver before restarting the service.

```sh
sudo systemctl stop powerglove-receiver.service
sudo /opt/powerglove/bin/powerglove-receiver --listen 0.0.0.0 --token-file /etc/powerglove/token --dr
sudo systemctl start powerglove-receiver.service
```

Start one-time-code pairing

Use `/opt/powerglove/bin/powerglove-pair` on RetroPie with `sudo`. It opens a temporary TLS server, prints a code, installs the received token, and restarts the receiver. Complete the browser steps while it is running.

Flag	Default	Meaning
`--listen ADDRESS`	`0.0.0.0`	Local IPv4 address for the temporary server.
`--port NUMBER`	`55357`	Pairing server TCP port. The browser pairing client uses the standard port; keep the default for that workflow.
`--token-file PATH`	`/etc/powerglove/token`	Destination for the paired token; keep it aligned with the receiver's token file.

Flag	Default	Meaning
`--timeout SECONDS`	`120`	Lifetime of the pairing server. Use a positive value; the code is single use and attempts are limited.
`--receiver-service NAME`	`powerglove-receiver.service`	systemd service to restart after pairing succeeds.
`-h`, `--help`	-	Prints usage and exits.

The command returns `0` after pairing completes or `2` when the pairing window times out. Other failures are reported as errors.

Select a profile from RetroPie

Use `/opt/powerglove/bin/powerglove-profile` to test profile selection. It sends an authenticated request to the PowerGlove Vision Controller. This changes the active profile; it does not change the saved startup profile or turn controller delivery on.

Flag	Default	Meaning
`--uno-q HOST`	Required	PowerGlove Vision Controller hostname or IPv4 address.
`--port NUMBER`	`55356`	PowerGlove Vision Controller profile-control UDP port.
`--token VALUE`	None	Supplies the token directly; prefer the protected-file option.
`--token-file PATH`	None	Reads the token from a file. Supply exactly one token option.
`--registry PATH`	`/etc/powerglove/games.json`	Registry used when `--profile` is omitted.
`--system NAME`	`nes`	System used for registry selection and request metadata. Automatic selection accepts `nes` or `famicom`.
`--rom PATH`	`Manual selection`	ROM path or filename used for registry lookup and displayed game metadata.
`--profile NAME`	None	Overrides registry lookup. Accepts `program_a` through `program_i`, `bad_street_brawler`, `super_glove_ball`, or `off`.
`--timeout SECONDS`	`0.4`	Wait for each acknowledgement; the sender makes up to three attempts.
`-h`, `--help`	-	Prints usage and exits.

Exit codes are `0` for acceptance, `2` for a timeout, and `3` for rejection. Acceptance means the request is queued; confirm the applied profile on Dashboard. Malformed arguments and file errors can also stop the command.

Run the paired Games service

The RetroPie installer starts this service automatically. Its installed command is `powerglove-games`; developers can also run `python3 -m powerglove_vision.game_registry`.

Flag	Default	Purpose
`--settings`	`/etc/powerglove/launcher.json`	Read the administrator-configured registry and token-file locations.
`--listen`	`0.0.0.0`	Bind address for the paired LAN service.
`--port`	`55358`	TCP port. The UNO browser proxy uses the standard port.
`--help`	Not applicable	Print usage without starting the service.

The service uses a single request handler with a three-second socket timeout, 64-KiB registry limit, and 64 pending challenges. Challenges expire after fifteen seconds and can be used once. The previous registry is saved beside the active file as `games.json.previous` (or the configured filename plus `.previous`). Installer-generated systemd permissions allow writing to the configured registry directory. Rerun installation if an administrator changes that directory.

Forward game-launch events

The installer connects `powerglove-retropie-hook` to RetroPie automatically. Use its wrapper scripts from existing runcommand hooks; do not run them again as an extra installation step.

Argument or flag	Default	Meaning
`ACTION`	Required	`start` creates a registered-game session; `end` turns gestures off. The internal `session` action is reserved for the detached heartbeat process.
`SYSTEM`	Empty	Optional first metadata argument, such as `nes`.
`EMULATOR`	Empty	Optional second metadata argument supplied by RetroPie; accepted for compatibility.
`ROM`	Empty	Optional third metadata argument; its filename selects the profile.
`COMMAND`	Empty	Optional fourth metadata argument; accepted for compatibility. Quote it as one argument.
`--settings PATH`	`/etc/powerglove/launcher.json`	Reads destination, token path, registry path, and timeout settings.
`--session-file PATH`	Current user's `.cache/powerglove-vision/active-game.json`	User-owned marker that lets a newer launch or the end hook invalidate an older monitor. Normally set only by the installed wrapper.
`--heartbeat-seconds NUMBER`	`2.0`	Renewal interval while the registered session and RetroArch are both active. Allowed range: 0.25-5 seconds.
`--lease-seconds NUMBER`	`6.0`	PowerGlove Vision Controller validity window for each renewal. Allowed range: 2-15 seconds.
`--startup-wait NUMBER`	`20.0`	Maximum wait for RetroArch to appear before abandoning the session. Allowed range: 1-60 seconds.
`-h`, `--help`	-	Prints usage and exits.

`retropie/runcommand-onstart-powerglove.sh` forwards RetroPie's four positional arguments. `retropie/runcommand-onend-powerglove.sh` needs none. Neither wrapper has its own flags. The executables in `retropie/bin/` forward their arguments to the corresponding Python commands. Update both the PowerGlove Vision Controller and RetroPie from the same release; an older hook cannot renew a session understood by the newer worker, and an older worker cannot apply the new lease fields.

Run the standalone vision tracker

`powerglove-vision` is the advanced camera worker, not the full App Lab website. Install the package's `vision` dependencies in a compatible Python environment before using it. Normal PowerGlove Vision Controller use should start through App Lab instead.

Flag	Default	Meaning
`--receiver HOST`	Required	RetroPie hostname or IPv4 address. An empty string permits local-only tracking but cannot deliver controls.
`--port NUMBER`	`55355`	Destination UDP controller port.
`--token VALUE`	None	Legacy direct shared token; exposes the value in process arguments. Prefer a private-file option.
`--token-file PATH`	None	Read the shared token from a private text file.
`--device-config PATH`	Supervisor: `data/device.json`	Read the shared token from private device JSON. Supply exactly one of the three token options. Other worker settings still come from their flags.
`--profile NAME`	`bad_street_brawler`	Initial profile, one of the eleven supported profiles or `off`.
`--camera VALUE`	`auto`	Camera selection; use `auto` or a camera index.
`--width PIXELS`	`640`	Requested capture width; the camera may negotiate another size.
`--height PIXELS`	`480`	Requested capture height.
`--fps NUMBER`	`0` (Automatic)	`0` tries 30 fps first and then accepts the driver-selected rate if necessary; explicit choices are 30 or 60. A requested rate is not a guarantee of effective tracking or game frame rate.
`--camera-buffers NUMBER`	`1`	One or two capture buffers; the measured UNO Q Kiyo candidate uses two.
`--kiyo-hdr-off`	Off	Identity-checked volatile Kiyo Pro HDR-off with automatic fixed-rate exposure.
`--camera-format VALUE`	`MJPG`	Requested V4L2 format, either `MJPG` or `YUYV`. Keep `MJPG` for normal use; compare both only with the performance readings on hardware that advertises them.
`--capture-backend VALUE`	`opencv`	`opencv` is the compatible reader. `direct-v4l2` is the optional newest-driver-buffer experiment and falls back to OpenCV when unsupported.
`--camera-exposure VALUE`	`auto`	`auto` makes no exposure changes; `low-latency` uses only advertised standard V4L2 controls; `kiyo-low-latency` adds the USB-identity-checked Kiyo HDR-off request; `manual` applies capability-checked exposure and gain through Direct V4L2.

Flag	Default	Meaning
`--camera-manual-exposure NUMBER`	`78`	Requested manual exposure. Used only with `--camera-exposure manual`; the active camera's advertised range and step remain authoritative.
`--camera-manual-gain NUMBER`	`96`	Requested manual gain. Used only with `--camera-exposure manual`; the active camera's advertised range and step remain authoritative.
`--inference-threads NUMBER`	`4`	CPU threads requested for each MediaPipe Hands inference calculator; accepted values are 1, 2, and 4. The Controller supervisor passes its validated setting explicitly. Benchmark before changing.
`--tracking-confidence NUMBER`	`0.35`	Minimum MediaPipe landmark-tracking confidence. This threshold matched or slightly improved the saved-clip result without changing the fast-sweep loss pattern; do not treat it as position confidence.
`--tracking-roi-scale NUMBER`	`2.25`	Scale of MediaPipe's next-frame hand search area. The prior `2.0` remains accepted for comparison; `2.25` recovered five of nine previously missed fast-sweep frames without a material latency or false-activation cost.
`--motion-tracking`	Ignored	Hidden compatibility spelling retained for old launch scripts; optical flow is archived and this flag does not enable it.
`--tracker-backend VALUE`	`legacy`	`legacy` selects **MediaPipe Hands**; `tasks-video` selects **MediaPipe Tasks Video (experimental)** using the packaged Hand Landmarker model. The identifiers remain stable for scripts.
`--tracker-graph VALUE`	`full`	`full` retains the proven outputs. `lean-image` omits world-landmark and handedness output streams for an output-paused comparison.
`--preview-fps NUMBER`	`5`	Maximum rate at which optional browser preview jobs are submitted.
`--glove-color VALUE`	`none`	`none`, `white`, or `black`; an informational label, not a different recognition model.
`--no-mirror`	Off	Disables horizontal image mirroring.
`--config PATH`	Project `config/profiles.json`, if present	Alternative gesture-threshold file. Otherwise built-in defaults are used.
`--model PATH`	Default verified model	Alternative MediaPipe Tasks model used only with `--tracker-backend tasks-video`. The standard `hand_landmarker.task` filename uses the verified model workflow.
`--web-host ADDRESS`	`0.0.0.0`	Address for the worker's diagnostic web server.
`--web-port NUMBER`	`8088`	Worker diagnostic port. App Lab overrides this to `8089` on loopback behind its main web server.
`--no-matrix`	Off	Disables direct matrix integration. App Lab uses this because its supervisor controls the matrix.
`--profile-listen ADDRESS`	`0.0.0.0`	Local IPv4 address for authenticated profile requests.
`--profile-port NUMBER`	`55356`	UDP port for profile requests.
`--controller-enabled`	Off	Starts sending controller input immediately; omit it for local inspection.
`-h`, `--help`	-	Prints usage and exits.

Collect a live status baseline

Run `python3 scripts/measure-vision-status.py` from the development checkout. The command performs GET requests only. It does not open the camera, arm the controller, change profiles, or record images. Set the intended state through Dashboard before starting. See the baseline procedure for interpretation and the separate receiver, core, and display measurements.

Flag	Default	Meaning
`--status-url URL`	Required	Explicit HTTP(S) `/status` endpoint without credentials, query, or fragment. Ordinary environment proxies are ignored. HTTPS uses normal certificate validation.
`--seconds NUMBER`	`30`	Observation duration, from 1 to 600 seconds. Requests have a timeout of at most two seconds.
`--interval NUMBER`	`0.1`	Seconds between requests, from 0.05 to 5. Slow requests reduce the actual poll rate.
`--phase NAME`	Required	`neutral`, `movement`, or `idle`. Neutral adds aggregate X/Y span and standard deviation for detected, calibrated samples; the label is supplied by the operator, not inferred from the hand.
`--output PATH`	Required	New JSON report path. The parent directory must exist; existing reports are never overwritten. Keep reports outside release files.
`-h`, `--help`	-	Prints usage and exits.

Exit `0` means at least one fresh active sample was observed, not that a latency target passed. Exit `2` means no active samples or invalid arguments. Inspect request errors, detection/calibration counts, send-success counts, and condition segments before comparing runs. Timing distributions describe unique observed samples; public status polling can miss intervening inference results. Reports separate detected-hand from missing-hand inference costs and include the skipped-capture counter delta within each condition segment.

Deploy or repair the PowerGlove Vision Controller application

Run these scripts from the project checkout on your development computer. Always supply your own SSH destination; the built-in fallback names the maintainer's board and is not appropriate for other installations.

Script or setting	Arguments and defaults	Effect
`scripts/deploy-uno-q-wifi.sh`	Optional positional `USER@HOST`; `-h` or `--help`; optional `UNO_Q_SSH_IDENTITY` private-key path	Transfers the Linux application, preserves `data/`, restarts the container, and checks web routes. Does not update RetroPie or flash the matrix sketch.
`UNO_Q_SSH_TARGET`	Environment variable; overridden by a positional destination	Sets the SSH destination. Without either setting, deployment falls back to the maintainer's board.

Script or setting	Arguments and defaults	Effect
`UNO_Q_APP_DIR`	Environment variable; default `/home/arduino/ArduinoApps/powerglove-vision`	Remote deployment directory. Changing it does not change the shutdown helper's fixed path or the machine installer's path requirement.
`scripts/install-uno-q-shutdown-helper.sh`	Optional positional `USER@HOST`; `-h` or `--help`	Installs the fixed shutdown watcher, service, and readiness rule. Uses the positional destination, then `UNO_Q_SSH_TARGET`, then the maintainer's fallback. The application directory is fixed.

Both scripts accept at most one destination. Deployment needs existing SSH key access; helper installation can prompt for a password through the terminal.

Build, inspect, or maintain project files

These scripts run from a full development checkout. Scripts marked **no flags** do not implement `--help`; adding it is not a safe way to inspect them because they may still perform their normal work.

Script	Arguments and flags	Result or requirement
`scripts/build-app-lab-package.sh`	No flags or positional arguments	Builds `output/app-lab/PowerGlove-Vision-Uno-Q.zip`; requires Bash, rsync, zip, and the existing public PDFs.
`scripts/verify-app-lab-package.py`	Optional `ARCHIVE` path; `-h`, `--help`	Checks the supplied ZIP or the default ZIP above; prints its SHA-256. Returns `0` on success, `1` on verification failure.
`scripts/check-documentation.py`	`--require-pdfs`; `-h`, `--help`	Checks Markdown, links, and coverage. The optional flag also inspects the PDF set and needs `pypdf`. Returns `0` on success, `1` on failure.
`scripts/check-source-docs.py`	No flags or positional arguments	Checks source headers and docstrings; returns `0` on success or `1` on failure.
`scripts/build-docs-pdf.py`	No flags or positional arguments	Rebuilds all registered PDF editions; requires ReportLab. Use only when ready to regenerate the PDFs.
`scripts/build-nestopia-powerglove.sh`	Optional build-directory positional argument	Clones a pinned official Nestopia revision, applies the isolated research patch, and builds a separately named core. It does not install or promote the core.
`scripts/build-fceumm-benchmark.sh`	Optional build-directory positional argument	Builds a pinned stock FCEUmm core in an isolated directory for the direction-response comparison. It does not install the core.
`scripts/install-nestopia-powerglove.sh`	Optional build-directory positional argument	Run with `sudo` on RetroPie after exact-ROM validation. Builds and installs only `lr-nestopia-powerglove`, plus its upstream GPLv2 license and distribution note; stock Nestopia remains untouched. The normal RetroPie installer offers this step when a registered Super Glove Ball ROM is found.

Script	Arguments and flags	Result or requirement
`scripts/configure-super-glove-ball-core.py`	`--rom PATH --mode MODE [--apply]`, where MODE is `native` or `fceumm`	Previews or atomically selects the custom core for one Super Glove Ball ROM. `--mode fceumm` is the explicit rollback.
`scripts/run-nestopia-powerglove-trace.py`	Core, exact ROM, trace/state/scratch paths	Runs controlled native phases, records the ROM digest and packet evidence, and can save temporary validation frames.
`scripts/benchmark-direction-response.py`	Paths to both cores and the exact Super Glove Ball ROM, scratch path, optional FCEUmm reference ROM, frame count, and JSON output	Runs matched-savestate activation and release comparisons for the same ROM in native and FCEUmm modes. The optional reference lane uses Gun Smoke. ROMs and scratch output remain outside the project.
`scripts/record-vision-benchmark.py`	Optional camera, output, size, and frame-rate flags	Records a fixed 30-second, local-only cue sequence for near/far recognition, X/Y travel, jitter, depth, and recovery comparisons. It is never run by installation or used for training.
`scripts/guided-vision-benchmark.py`	Optional camera, output, bind address, port, size, and frame-rate flags	Serves a temporary live-preview page for user-paced, per-step benchmark recording. Each selected step has a two-second countdown; pauses between steps are not recorded. The camera is released when capture completes. Output stays local and is not training data.
`scripts/benchmark-vision-replay.py`	Local clip, required JSON output, and optional Tasks model path	Replays the same full frames through MediaPipe Hands at 1, 2, and 4 threads and through optional Tasks Video, at 640×480 and full-field 512×384, with preview closed and open. Reports p50/p95 inference, continuity, cue recognition, neutral false activations, coordinate jitter, and preview cost.
`scripts/benchmark-native-motion-curve.py`	Version-2 vision replay JSON, optional lane index, and required new output path	Compares the former overshooting experiment, capped error curve, actual bounded speed curve, and unsmoothed coordinates. Sweeps 27 bounded candidates and reports jitter, lag, medium response, fast pickup, reversals, overshoot, continuity, and available source age without controlling a game.
`scripts/benchmark-camera-pipeline.py`	Required `--camera DEVICE` and `--worker-stopped`; optional `--source-root PATH` and `--seconds 5..30`	Linux-only, output-paused capture/recognition diagnostic. Requires exclusive camera ownership, compares one/two/one V4L2 buffers, performs fixed-frame profiling, keeps images in memory, and prints progress plus the final numeric report to standard output. It does not change camera controls or player settings.
`scripts/benchmark-palm-anchors.py`	Version-2 replay JSON and required new output path	Compares the five-point baseline, four-knuckle centroid, palm-polygon center, and weighted wrist/knuckle center for pose shift, travel retention, continuity, and reacquisition. It reports evidence but does not change the live anchor.
`scripts/benchmark-frame-preprocessing.py`	Camera or clip input and required new output path	Output-paused comparison of mirrored-frame preparation and reusable buffers. It cannot change handedness or preview conventions.

Script	Arguments and flags	Result or requirement
`scripts/benchmark-staggered-trackers.py`	Camera or clip input and required new output path	Isolated two-tracker newest-sequence experiment. It never arms controller output and is not a gameplay backend.
`scripts/benchmark-tasks-live-stream.py`	Camera or clip input, Tasks model, delegate choice, and required new output path	Isolated MediaPipe Tasks live-stream CPU/GPU probe with one result in flight and newest-sequence accounting. GPU support and performance must be demonstrated on the target; this is not a production mode.
`scripts/analyze-motion-trace.py`	Required trace path; optional `--output NEW-PATH`	Reads one finite controller motion trace and reports recognition source age, selected-versus-filtered error, movement-class settling, fallback reasons, and tracking losses. Without `--output`, JSON is printed; an existing output file is never overwritten.
`scripts/compare-motion-matrix.py`	Required directory and `--output NEW-PATH`	Compares `min*-boost*.trace.json` files using the shared analyzer. Use only for windows with the same movement sequence and camera conditions; the output file must not already exist.
`scripts/benchmark-motion-correction.py`	Required `--before PATH` and `--output NEW-PATH`; optional `--after PATH`	Synthetic before/after benchmark for the motion-correction implementation. It uses generated frames and simulated recognition delay, never a camera or game, and cannot establish physical latency.
`scripts/analyze-motion-samples.py`	Required `--samples-dir PATH` and `--output NEW-PATH`	Summarizes saved aggregate status samples and models ideal native-coordinate steps through the actual smoothing engine. It does not replay input or measure physical latency; the output file must not already exist.
`scripts/calibrate-reach.py`	One required step: `begin`, `center`, `left`, `right`, `up`, `down`, `apply`, or `cancel`	Internal operator helper for the guided comfortable-reach procedure. Run one step at a time inside the Controller container as described above; it pauses output and preserves a private backup.
`scripts/measure-dot-input.py`	Optional `--state PATH`, `--seconds NUMBER`, and `--interval NUMBER`; required `--output NEW-PATH`	Reads the cabinet's native-state record without changing it and reports dot validity, loss/recovery, distinct publications, and coordinate range. Defaults are the installed state path, 30 seconds, and 60 polls per second.
`scripts/configure-uno-q-avahi.py`	Optional `--config PATH` and `--interfaces NAME...`	Internal host-installer helper. It restricts Avahi to validated physical interfaces, preserves a backup, and defaults to detecting interfaces from Linux sysfs; ordinary users should rerun installation instead.
`scripts/build-gesture-crops.py`	No flags or positional arguments	Regenerates action illustrations from the gesture sheets; requires Pillow.
`scripts/fetch-runtime-assets.sh`	No flags or positional arguments	Installs and verifies the bundled model into project `data/models/`; downloads only if absent. Requires Python 3, plus curl for fallback downloads.

Script	Arguments and flags	Result or requirement
`scripts/configure-uno-q-mdns.py`	Required positional path to the generated Compose file; no flags	Internal installer/deployment helper that edits that file. Prefer the supported setup command.
`scripts/profile-relay.py`	No flags or positional arguments	Internal unprivileged UDP relay; publishes port 55356 and forwards to the worker, with bounded packet size, outstanding requests, and timeouts.
`scripts/avahi-resolver-service.py`	No flags or positional arguments	Internal service started by the resolver brick; opens its fixed Unix socket.
`python/main.py`	No flags or positional arguments	App Lab entry point; reads `data/device.json` and supervises the worker.
`python/ssh_pair.py`	No command-line flags	Internal password-pairing helper; receives its request through standard input. Use secure Setup instead.

Understand the system commands in these guides

The table explains the subcommands and flags shown in the documentation. Defaults and additional flags for these external programs depend on their installed versions; use `man TOOL` or that tool's `--help` for its full manual.

Command or syntax	Meaning of the options used here
`git clone --branch dev URL`	Downloads a repository and checks out `dev`. A final directory argument chooses the destination folder.
`git branch --show-current`	Prints the checked-out branch name.
`git status --short`	Shows a compact list of local changes.
`git pull --ff-only`	Updates only if Git can fast-forward; refuses an automatic merge when histories have diverged.
`cd PATH`, `cd ~`, `pwd`	Changes directory, opens your home directory, or prints the current directory.
`command -v NAME`	Prints where the shell finds a command; no path means it is unavailable.
`sudo COMMAND`	Runs a command with administrator privileges. Enter your account password only at the terminal prompt.
`apt update`; `apt install -y PACKAGES`	Refreshes package information, then installs packages. `-y` accepts package-manager confirmation prompts.
`python3 -m MODULE`	Runs a Python module. Used with `venv`, `pip`, `unittest`, `json.tool`, and `compileall`.
`python3 -m venv DIRECTORY`	Creates an isolated Python environment at that directory.
`. .venv/bin/activate`	Activates that environment in the current shell. The initial dot is a shell command.
`pip install -e PATH`	Installs a source checkout in editable mode. `[vision]` and `[receiver]` request optional dependency groups.

Command or syntax	Meaning of the options used here
`pip install -U PACKAGE`; `--upgrade`	Upgrades the named package; the two options have the same meaning.
`unittest discover -s tests -v`	Finds tests in `tests` (`-s`) and prints each test's result (`-v`).
`compileall -q DIRECTORIES`	Checks/compiles Python files; `-q` reduces routine output.
`json.tool FILE`	Parses and formats JSON; a parse error means the example or configuration is invalid.
`PYTHONPATH=src COMMAND`	Sets the import path for that command only, allowing it to use the local `src/` tree.
`ssh-keygen -t ed25519`	Creates an SSH key pair using the Ed25519 algorithm (`-t`). Follow the prompts without overwriting an existing key.
`cat FILE`	Prints a text file. In key setup, print only the `.pub` public key.
`ssh USER@HOST`	Opens a remote terminal. `exit` closes that session.
`ssh -o BatchMode=yes USER@HOST COMMAND`	Runs a remote command without interactive password prompts; useful for checking key access. `-o` supplies an SSH setting.
`ssh -t USER@HOST COMMAND`	Allocates a remote terminal, allowing an interactive remote prompt.
`docker port CONTAINER 55356/udp`	Shows the host binding for the container's UDP profile port. This command does not change the container.
`systemctl status NAME`	Displays a service or timer's state. Press `q` if it opens a pager.
`systemctl start`, `stop`, `restart NAME`	Starts, stops, or restarts the named unit now.
`systemctl enable`, `disable NAME`	Enables or disables activation at boot. `--now` also starts or stops it immediately.
`systemctl is-enabled`, `is-active NAME`	Checks boot enablement or current activity.
`systemctl daemon-reload`	Reloads unit definitions after service files change.
`systemctl --no-block halt`	Requests a halt and returns without waiting. This is the helper's action, not a health check.
`journalctl -u NAME -n 100 --no-pager`	Shows logs for a unit (`-u`), limits output to the last 100 entries (`-n`), and avoids a pager.
`systemd-tmpfiles --create FILE`	Applies creation rules from the named file, such as the readiness-marker rule.
`grep -A8 -B2 PATTERN FILE`	Shows matching lines with eight following lines (`-A`) and two preceding lines (`-B`).
`curl -sS URL`	Fetches a URL quietly (`-s`) while still showing errors (`-S`).
`ls -l PATH`	Lists files with permissions and ownership. A trailing directory slash refers to its contents.
`install -d -m MODE PATH`	Creates a directory (`-d`) with the specified permissions (`-m`). Without `-d`, `install` copies a file; `-o` sets its owner and `-g` its group.
`chmod MODE PATH`; `chown USER:GROUP PATH`	Changes permissions or ownership. Modes used here: `0600` owner only, `0640` owner read/write and group read, `0644` public read with owner write, `0700` private directory, `0755` publicly traversable directory or executable.
`rm FILE`; `rm -f FILE`	Deletes a file; `-f` suppresses prompts and ignores a missing file. Check the path before use.

Command or syntax	Meaning of the options used here
`nano FILE`	Opens a text editor. Save with Ctrl+O, confirm the name, and exit with Ctrl+X.
`modprobe uinput`	Loads the Linux virtual-input kernel module.
`tee FILE`	Writes its input to a file as well as the terminal. Used with `sudo` for a protected destination.
`printf '%s\n' VALUE`	Prints a value followed by a newline.
`ping HOST`	Tests whether a host replies to network echo requests; press Ctrl+C to stop. It does not test the application.
`hostname`; `hostname -I`	Prints the host's name, or its interface addresses with `-I`.
`bash -n FILE`; `sh -n FILE`	Checks shell syntax without running the file.
`> /dev/null`	Discards normal output while leaving errors visible.
Pipe operator	Sends the first command's output into the second command. The vertical bar is shell syntax.
`\` at the end of a command line	Continues the same command on the next line.
`"$1"` through `"$4"`	Quoted positional arguments forwarded by RetroPie's wrapper scripts.
`for ...; do ...; done`	Repeats the enclosed commands for each value.
`[ -r FILE ]`	Tests whether a file is readable. `continue` skips the current loop item.
`${device##*/}`	Extracts the last component of a path in the camera-inspection loop.

Commands shown inside a source script may use additional internal options. Those are implementation details, not extra arguments accepted by the wrapper.

Tuning measurements and interface contract

Each three-second step needs at least twelve accepted measurements. Sampling requires a calibrated, detected hand with finite, non-collapsed palm geometry; duplicate frames and non-finite measurements are excluded. Missing or invalid tracking does not contribute samples. MediaPipe Hands' score remains labelled as handedness certainty rather than being used as position confidence. Insufficient samples require retrying the step. Changing neutral calibration clears recordings and preview values.

Suggestions use the 95th percentile of the two open/rest recordings and the 10th percentile of the performed recording. The gap must be at least 0.08; activation is placed 65% into that gap and release 30% into it. An overlap error identifies the affected component, and no suggestion is retained. Fully return to the starting pose and distance before recording the final step.

For V-sign and thumbs-up, extended-only fingers retain existing thresholds; these samples cannot establish their curled boundary. Personal activation values check curled fingers and personal release values check extended fingers. Without personal settings, the legacy menu cutoffs in the table above apply. Live feedback and actual pose recognition use the same checks, with the existing deliberate hold retained. Before accepting an automatic suggestion, the app applies the candidate thresholds to all required fingers in the recordings. At least 90% of accepted samples must match the complete gesture together; the first and final recordings must likewise show all selected fingers extended. This allows limited tracking noise without letting a strong curl compensate for a bent extended finger. A failed check identifies the finger and phase and clears the preview.

V-sign requires extended index/middle and curled ring/pinky; thumbs-up requires an extended thumb and four curled fingers. Thumb extension measures straightness, not direction toward the top of the image. Individual tuning keeps extended-only thresholds; use Set up my hand if comfortable extension needs adjustment. Preview and live gameplay testing still matter, and manual numerical edits remain a separate path rather than an automatically verified recording.

The status response's `tuning` object retains its version-1 fields and adds `problem`, `wizard_step`, `recipe`, `stable_ready`, `test`, `image_quality`, and `diagnostic`. `gesture`, `total_phases` (3), `completed_phases`, and `finger_feedback` continue to describe the active recipe. Each finger entry has `expected` (`extended` or `curled`), `matches`, `value`, and `threshold`. `saved`, `effective`, and `preview` distinguish persisted values, currently used pairs, and temporary adjustments. The six-second browser lease expires temporary recordings and previews; it does not delete saved thresholds. Manual preview and selective reset remain available under Advanced; normal wizard saving requires the guided activation, release, and neutral check.

Verified sketch dependencies

`sketch/sketch.yaml` pins Arduino Zephyr **1.0.0** for `arduino:zephyr:unoq`, Arduino_RouterBridge **0.4.3**, Arduino_RPClite **0.3.0**, ArxContainer **0.7.0**, ArxTypeTraits **0.3.2**, DebugLog **0.8.4**, and MsgPack **0.4.2**. Keep this complete configuration synchronized with the installed app. Platform installation, compile-only validation, and firmware upload are separate operations; see Build and install matrix firmware.

Bad Street Brawler Glove Zap

The dedicated `bad_street_brawler` profile translates each forward-push activation into simultaneous Left + Right for 180 milliseconds. It suppresses other D-pad directions and A/B during that pulse. Holding the push does not repeat it; cross the saved push Release threshold before activating again. Menu poses, tracking loss, and recalibration cancel the pulse. Other game profiles are unchanged. The game, rather than the gesture engine, enforces its once-per-round allowance.

FCEUmm normally removes opposing directions. For this game, set `fceumm_up_down_allowed = "enabled"` in its game-specific options file. On the cabinet this is `/opt/retropie/configs/all/retroarch/config/FCEUmm/Bad Street Brawler (USA).opt`. The filename omits the ROM/archive extension. Preserve any existing game options; when creating the file, start from the current FCEUmm options to avoid changing unrelated sound, video, or input settings. Leave the global value unchanged.

In RetroArch, the equivalent workflow is to load the game, open Quick Menu's Core Options, allow opposing directions, and save a Game Options file. Verify that the game-specific file is loaded on the next launch. The existing receiver and PowerGlove gamepad mapping already carry separate Left and Right buttons; no R2/extra-trigger binding is needed. Disable the option for this game or remove its new options file to undo that part of the change.

The secret Left + Right attack is documented by the runner in Speed Demos Archive. FCEUmm's input code shows the opposing-direction filter. Headless startup can verify options loading; confirm the attack, normal movement, and once-per-round behaviour in live play.

Repeatable RetroPie check and setup

The installer copies `scripts/configure-bsb-zap.py` and installs the `powerglove-bsb-zap` command. For an existing installation, copy the helper into `/opt/powerglove-src/scripts/` and its matching wrapper from `retropie/bin/` into `/opt/powerglove/bin/`. This is a separate, explicit game setup step; the main installer does not change game options automatically.

Run on RetroPie, using the exact ROM/archive path installed on your system:

```
SH
/opt/powerglove/bin/powerglove-bsb-zap --check \
  --rom "/home/pi/RetroPie/roms/nes/Bad Street Brawler (USA).7z"
```

To make the change, exit any running RetroArch game first:

```
SH
sudo /opt/powerglove/bin/powerglove-bsb-zap --apply \
  --rom "/home/pi/RetroPie/roms/nes/Bad Street Brawler (USA).7z"
```

Without `--apply`, the command is read-only. Exit code 0 means the checked configuration is ready; 2 means action is required. The helper checks the ROM, installed RetroArch and FCEUmm files, NES default and current/legacy per-game emulator selections, and explicit disabling of automatic game options. If FCEUmm is missing, install `lr-fceumm` through RetroPie Setup. If another core is selected, choose `lr-fceumm` for this ROM in the launch menu. The helper does not install emulators or change emulator selections.

Existing game options are preserved. A new file copies the first available options set in game, content-folder, core, then global order; only FCEUmm entries are copied from the global file. Only `fceumm_up_down_allowed` is changed. Updates are atomic, preserve existing file ownership/permissions, and create a `powerglove-bsb-backup-*` directory beside the options file with `RESTORE.txt`. Follow that file to restore the previous options or remove a newly created file. Running the helper again when enabled makes no changes or extra backups.

The default layout is `/opt/retropie`. `--prefix` supports a different installation root, and `--options-dir` specifies a different FCEUmm options directory. Custom core-option path redirects are rejected for manual review. This checks standard RetroPie files; it does not launch the core, resolve arbitrary custom launch scripts/includes, or prove the game loaded the option. Relaunch Bad Street Brawler, confirm its game-specific options are loaded, then test Glove Zap and normal movement. RetroArch documents game options as complete option sets, which is why the helper preserves the inherited settings instead of writing only one option.

Matrix startup hourglass

After the system boot display, the Arduino sketch draws a pulsing hourglass before connecting to Router Bridge. Its display task runs independently of the bridge connection and Python imports, so a slow app startup does not leave the matrix blank once the sketch is running. Loading uses five frames, 220 ms each. The supervisor requests loading before importing the web controls; its normal status updates then select the glove attract animation, profile, or Academy mode. This is an activity indicator, not a percentage or a guarantee of successful startup. If the app never connects, the hourglass continues.

The earlier protected boot display and any interval before the sketch starts belong to the board's system software. This change does not replace those. The display task uses the current Arduino sketch platform's thread API; if its stack cannot be allocated, the initial hourglass remains visible and ordinary loop animation resumes after bridge initialization. Verify a cold boot on the physical matrix, especially the transition from the system display to the hourglass.

Early-start host helper

The helper runs on every boot as the Arduino user, separately from App Lab. It reduces the blank interval after the system animation by releasing the installed sketch early. It preserves the existing hourglass and uses no reset, halt, flash, or replacement of Arduino system services. The one-boot trial was confirmed on the physical board before enabling repeated startup.

The UNO installer performs this automatically. For manual repair, install `scripts/uno-q-early-start.py` at `~/.local/lib/powerglove/uno-q-early-start.py` and `uno-q/powerglove-early-start.service` at `~/.config/systemd/user/powerglove-early-start.service`. The Arduino user must have lingering enabled.

Run `systemctl --user daemon-reload` and `systemctl --user enable powerglove-early-start.service`. Disable the old `powerglove-early-start-trial.service` if present. No armed marker is required.

The service waits up to 30 seconds for the router and permits 20 seconds for the debug check and release. It verifies the PowerGlove Vision Controller, PowerGlove startup app, Wait for App image header, and four 64-byte code samples. This is not a full integrity check. If the image is unavailable or differs, or the debug pins are busy, it fails without releasing the sketch; normal App Lab startup continues. App Lab may subsequently reset the sketch during its ordinary upload.

Inspect `journalctl --user -b -u powerglove-early-start.service`; disable with `systemctl --user disable powerglove-early-start.service`. Review compatibility after platform updates: the tested loader is Arduino platform 1.0.0 with App Lab 0.13.0. The helper writes only the startup release word `0xCAFFEEEE` at `0x40036400` after checking board identity, startup-app selection, the cached Wait for App header, and four 64-byte samples of installed sketch memory. This is a bounded compatibility check, not complete firmware attestation. Do not run it during uploads or alongside another debugger.

For validation, reboot rather than judging the user-service restart alone. A successful cold boot shows the Arduino logo and heart, measures the blank interval, starts the project hourglass, and reaches the usual glove animation and controls without reconnecting the camera. A service log that reports `released` confirms the helper action; it does not by itself prove cold-boot timing or application readiness. If the checks fail, leave the helper disabled and allow ordinary App Lab startup to remain the fallback.

Matrix animation timing reference

These implementation details support maintenance and physical display checks. For display meanings and user actions, use the Matrix display guide.

Display	Timing and implementation
Startup hourglass	Five frames, 220 ms per frame; repeats while loading
Academy L / tuning T	Eight scan frames, 160 ms per frame, with trailing glow
Tracked profile	Alternates bright and dim every 360 ms
Error X	Alternates X and blank every 420 ms
Pairing	Nine steps of about 650 ms; ID, seven hexadecimal characters in groups of two with one final character, PN, then three digit pairs

The idle glove sequence draws an energy streak, brings the cuff and glove into view, curls and reopens the fingers, moves a spark along the glove, then pulses and pauses before repeating. Pairing temporarily takes display priority, normally for two minutes; a shutdown/off request can clear it sooner. Status precedence can keep a mode letter or idle animation visible instead of X.

Manual installation maintenance

PowerGlove Vision Controller updates over Wi-Fi

Use this route after the initial App Lab installation. It updates the Linux application; matrix firmware changes still need **Run** in App Lab.

Set up SSH key access once

1. On your development computer, check for an existing public key in `~/.ssh/`. Use only a file ending in `.pub`; never copy its matching private key.
2. If you do not have a key, run `ssh-keygen -t ed25519`. Accept the suggested location only if it does not replace an existing key, and follow the passphrase prompts.
3. Open your public-key file and copy its complete single line. For the default key, run `cat ~/.ssh/id_ed25519.pub`.
4. Connect with `ssh arduino@UNO-Q-NAME.local`. On the PowerGlove Vision Controller, run `install -d -m 0700 ~/.ssh`, then `nano ~/.ssh/authorized_keys`.
5. Add the public key on a new line, preserving any existing keys. Save with Ctrl+O, confirm the name, and exit with Ctrl+X.
6. Run `chmod 0600 ~/.ssh/authorized_keys`, then `exit` to return to your computer. If your private key has a passphrase, make it available through your computer's SSH agent before the unattended deployment check.
7. Run `ssh -o BatchMode=yes arduino@UNO-Q-NAME.local hostname`. Continue only when it prints the PowerGlove Vision Controller hostname without requesting a login password.

Update the application

1. On your development computer, open your project checkout and review local changes with `git status --short`.
2. If you are updating from GitHub, run `git pull --ff-only`. Resolve any reported local-change or branch conflict before deploying. Keep the version compatible with the RetroPie installation.
3. Run the deployment command below. It preserves private `data/`, restarts the application, and checks its web pages.
4. Open Dashboard, Play, and Glove Academy to confirm the updated app works. If you changed the matrix sketch, also rebuild and run it through App Lab.

```sh
scripts/deploy-uno-q-wifi.sh arduino@UNO-Q-NAME.local
```

Use the board's current IP address if its local name is temporarily unavailable. If SSH access fails, reconnect through App Lab before retrying deployment. For local-name failures, see hostname resolution.

Repair or update the shutdown helper

The helper requests `systemctl --no-block halt`, not `poweroff`: the PowerGlove Vision Controller can reboot after a power-off request. A halt requests that Linux stop; it does not disconnect electrical power. LEDs may remain lit. **Known limitation, confirmed September 3, 2026:** our PowerGlove Vision Controller restarted after reaching the halt target both through a powered USB-C hub and when connected directly to a Mac. Shutdown is therefore not a verified way to keep this board stopped. Do not treat the website disappearing as a safe-to-unplug indicator or rely on a fixed countdown. See Arduino shutdown guidance.

The App Lab container is intentionally unprivileged and cannot halt the Linux host. The machine installer already installs the fixed-purpose systemd path helper. To repair or update it separately, run this command from your development computer's project checkout:

```
SH
scripts/install-uno-q-shutdown-helper.sh arduino@UNO-Q-NAME.local
```

Enter the PowerGlove Vision Controller `arduino` account password at the remote `sudo` prompt. The script does not read or store it. The helper watches only the fixed `data/shutdown-request` path and can perform only a system halt. After it is installed, **Shutdown** is available on Dashboard and **Shut down Controller** on Setup. Each press requires browser confirmation and warns that the PowerGlove Vision Controller may restart automatically and that a disconnected website does not confirm it is safe to remove power. Its boot-time tmpfiles rule recreates the readiness marker if the PowerGlove Vision Controller reboots or App Lab replaces the application directory.

Verify the helper without triggering shutdown:

```
SH
ssh arduino@UNO-Q-NAME.local
systemctl is-enabled powerglove-system-shutdown.path
systemctl is-active powerglove-system-shutdown.path
exit
```

Expected results are `enabled` and `active`. Checks on the cabinet on September 3, 2026 confirmed that the watcher was enabled and active, the readiness marker existed, and both pages displayed the Shutdown button. Requests without confirmation were rejected. Do not test the accepted API path unless you intend to shut down the PowerGlove Vision Controller.

RetroPie updates

1. On RetroPie, back up customized files under `/etc/powerglove/`, especially `games.json` and `launcher.json`, using your normal private backup method.
2. Open the original source checkout, normally `~/PowerGlove-Vision`. The installed copy under `/opt/powerglove-src` is not a Git checkout.
3. Run the commands below. Review `git status --short` before pulling; if Git reports a conflict, resolve it before running the installer.
4. Resolve any **FAIL** in the installer report, then launch a registered game and check its profile and controls. The installer preserves existing settings and tokens.

```
SH
cd ~/PowerGlove-Vision
git status --short
git pull --ff-only
sudo python3 scripts/setup-machine.py retropie --peer UNO-Q-NAME.local
```

Duplicate App Lab entries

Importing a newer ZIP may create a timestamped copy. The supported host installer and shutdown helper require `/home/arduino/ArduinoApps/powerglove-vision`. Keep the working application at that path; do not run host setup from a duplicate. Use the Wi-Fi update procedure for routine Linux application changes. Before removing any duplicate in App Lab, confirm which copy has your private settings and keep only the intended application set to start at boot.

Installation troubleshooting commands

Matrix shows a blinking X

- Confirm that an active gesture profile is selected. **Gestures off** should display the animated glove attract sequence, never the error X.
- Confirm the camera is connected through the powered hub.
- Try another hub port or USB cable.
- Check whether Linux sees a USB camera; internal `qcom-venus-encoder` and `qcom-venus-decoder` nodes are codecs, not your camera.
- Restart the app after checking power and cabling.

Camera appears only after reconnecting it

This usually indicates USB enumeration or power trouble. Keep the powered hub energized before starting the PowerGlove Vision Controller, try another cable, and avoid passive adapters. The app first retries an ordinary camera reopen. With the standard host helper installed, a camera that remains absent for 15 seconds receives one guarded hub-reset attempt; USB Ethernet can disconnect briefly. A camera still absent afterward needs its physical connection checked.

First installation takes several minutes

The first launch downloads and prepares the software it needs. Keep the PowerGlove Vision Controller connected to the internet and watch App Lab for progress. Later launches reuse the installed software and normally start faster.

Setup page does not open

- Ordinary settings: `http://UNO-Q-NAME.local:8088/setup`
- Secure pairing: `https://UNO-Q-NAME.local:8443/setup`
- Try the board's IP address if `.local` does not resolve.
- HTTPS and HTTP are not interchangeable on these ports.

Camera is slow to start or missing after reboot

The startup timer includes preparation as well as opening the camera. Allow more time just after a reboot and wait for the camera view before calibrating.

If the app reports that the camera is unavailable, run these commands on the PowerGlove Vision Controller:

```
SH
lsusb
ls -l /dev/v4l/by-id/
```

Look for your camera in both outputs. If it is absent from the USB list, check its cable and hub connection, then disconnect and reconnect it. Waiting for MediaPipe will not fix a camera that the operating system cannot detect. Keep Camera set to **Automatic - choose the connected camera** unless you intentionally selected a particular listed device. For stage timings and further checks, see Vision startup and timing.

Password pairing fails

- Prepare a new attempt and use its new matrix PIN.
- Confirm the RetroPie username and password can log in through SSH.
- The account must be allowed to run `sudo` with that password.
- Prefer the one-time-code method if password SSH is disabled.

Controller does not appear on RetroPie

```
SH
systemctl is-enabled powerglove-receiver.service
systemctl is-enabled powerglove-receiver.timer
sudo systemctl status powerglove-receiver.service
sudo systemctl status powerglove-receiver.timer
sudo journalctl -u powerglove-receiver.service -n 100 --no-pager
ls -l /dev/uinput
```

Allow 45 seconds after boot. Confirm `uinput` is loaded and `/etc/powerglove/token` is not empty. Expected boot enablement is `disabled` for the service and `enabled` for the timer. The virtual controller appears only after an authenticated packet arrives.

Frontend slowdown or USB-device conflicts

Confirm the receiver service was not enabled directly. Stop it, restart EmulationStation, and start the receiver afterward as an A/B test. If the frontend becomes responsive, restore the supplied timer. Also ensure Pixelcade's `game-select` and `system-select` directories contain only one executable hook each if you use a BitPixel display; executable backup scripts are additional hooks.

Controller exists but does not move

- Select **Start controller** on Setup or Debug.
- Confirm a calibrated hand and controller output on Debug.
- Verify the receiver address and UDP 55355 connectivity.
- Check whether an existing cabinet input merger filters the virtual device.

Profiles do not change

A **profile queued** message confirms authentication and queue admission. Wait for Dashboard to show the new profile; camera startup may still be in progress. For timeouts, check that the PowerGlove Vision Controller publishes UDP `55356` through its profile relay. Follow Check a queued profile change for the command and recovery steps. Check the exact ROM filename, including its archive extension, if the selected profile is **off**.

- Test `powerglove-profile` manually.
- Check `uno_q` and `token_file` in `/etc/powerglove/launcher.json`.
- Confirm both runcommand hooks call the supplied helper scripts.
- Match the exact ROM basename in `/etc/powerglove/games.json`.

FAQ: What if the console name cannot be resolved?

1. In **Connection**, enter your console's actual hostname, such as `RETROPIE-NAME.local`, then select **Check console address**. Use a hostname or IPv4 address, not `http://`, a port, or a page path. This tests resolution from the PowerGlove Vision Controller app; successful lookup on your laptop alone is not sufficient.
2. Confirm the RetroPie console is powered on and connected to your LAN. On its terminal, run `hostname` and `hostname -I` to confirm its name and current addresses. Do not assume an old DHCP address is still correct.
3. From the PowerGlove source directory on RetroPie, run `sudo python3 scripts/setup-machine.py retropie --check`. Check Avahi with `systemctl is-active avahi-daemon` and `systemctl is-enabled avahi-daemon`. If setup is incomplete, rerun `sudo python3 scripts/setup-machine.py retropie --peer UNO-Q-NAME.local`, using your board's actual name, and review every FAIL or ACTION result.
4. On the PowerGlove Vision Controller, from the app directory, run `sudo python3 scripts/setup-machine.py uno-q --check`. This checks the configured destination from inside the application. If installation is incomplete, rerun `sudo python3 scripts/setup-machine.py uno-q`. Do not manually patch `.cache/app-compose.yaml`: App Lab regenerates it. The shipped resolver brick supplies the persistent configuration.
5. Check that both machines are on a network that allows communication between devices. Guest Wi-Fi, client isolation, VPN routing, separate VLANs, or multicast filtering can prevent `.local` discovery. mDNS uses UDP port 5353; do not disable your firewall wholesale or expose the app to the Internet to fix discovery.
6. As a diagnostic or fallback, enter RetroPie's current LAN IPv4 address in **Connection** and test again. If that works while the name fails, investigate mDNS. For continued use, reserve that address in your router so DHCP does not change it. Save the intended destination using the normal Connection workflow; changing the address does not replace pairing credentials. If RetroPie also contacts the PowerGlove Vision Controller by name, check that reverse direction separately.
7. If neither name nor IP works, investigate connectivity and the service itself, not just Avahi. A successful name test only establishes name resolution; pairing, the receiver, controller output, and emulator mappings must also work. Retry after boot has finished, then collect the exact error and setup-check results if it still fails. Never share tokens, passwords, or private SSH keys.

After fixing the problem, reboot both machines and repeat **Check console address** before testing gameplay. The app-owned resolver has been verified across a PowerGlove Vision Controller reboot and a changed RetroPie DHCP address; no fixed IP entry is required for `.local` use.

Removing the installation

On RetroPie:

```
SH
sudo systemctl disable --now powerglove-receiver.timer
sudo systemctl disable --now powerglove-receiver.service
sudo rm /etc/systemd/system/powerglove-receiver.timer \
  /etc/systemd/system/powerglove-receiver.service
sudo systemctl daemon-reload
```

Remove only the PowerGlove lines from the runcommand hooks. After backing up custom profiles, `/opt/powerglove`, `/opt/powerglove-src`, and `/etc/powerglove` may be removed manually.

On the PowerGlove Vision Controller, stop the app, disable **Run at startup**, and remove it through Arduino App Lab. Its private `data` directory contains the device token and cached runtime.

Remove the host shutdown helper separately:

```
SH
sudo systemctl disable --now powerglove-system-shutdown.path
sudo rm /etc/systemd/system/powerglove-system-shutdown.path \
  /etc/systemd/system/powerglove-system-shutdown.service \
  /etc/tmpfiles.d/powerglove-system-shutdown.conf
rm -f /home/arduino/ArduinoApps/powerglove-vision/data/.shutdown-enabled
sudo systemctl daemon-reload
```

Build and install matrix firmware

The footer's **Software and matrix firmware** details show the candidate when available, exact software commit, modified-source indicator, and source SHA-256 read from the running matrix firmware. The expected fingerprint covers sketch sources and the pinned build profile; it is not a compiled-binary hash. Older firmware or an unavailable bridge reports **unavailable**. A differing readback indicates an update is needed. Expected metadata never substitutes for readback.

After editing sketch sources, run `python3 scripts/stamp-firmware-version.py` before compiling. Its `--check` option verifies `sketch/firmware_version.h`; packaging/deployment reject stale stamps. App Lab **Run** still uploads firmware. Readback runs in the supervisor every thirty seconds, outside the frame path.

To preview the idle animation from the actual sketch renderer on a development computer, install Pillow and a C++ compiler, then run `python3 scripts/build-matrix-animation-preview.py`. It writes `idle-animation.gif`, `idle-glove.png`, and `idle-storyboard.png` under `docs/images/matrix/` with the sketch's frame timings. Brightness is simulated, not calibrated to physical LEDs. This preview does not compile the Arduino firmware or upload anything.

The verified sketch profile is `arduino:zephyr:unoq` with Arduino Zephyr platform **1.0.0**. Keep the complete pinned `sketch/sketch.yaml`; installing a newer platform by itself does not rebuild or flash PowerGlove. A compile-only check builds the sketch but does not change the running firmware.

From the application directory on the PowerGlove Vision Controller, validate the pinned configuration:

```
SH
arduino-cli compile --profile default sketch
```

After a successful compile, use **Run** in App Lab to build and upload the sketch, or restart the app through App Lab's command-line tool:

```
SH
arduino-app-cli app restart /home/arduino/ArduinoApps/powerglove-vision
```

The Wi-Fi deployment script updates Linux files and restarts containers; it does not flash the matrix sketch. Keep a backup before firmware updates. After upload, check Dashboard, ordinary Glove Academy's scanning L, Tune's scanning T, and normal controller operation. Preserve the installed `data/` directory and verify that personal tuning and neutral calibration remain available.

Versioned two-script installation

The normal entry points are `scripts/install-uno-q.sh` and `scripts/install-retropie.sh`. Each resolves a published release tag, downloads its matching package and shared `install-package.py`, verifies SHA-256 checksums, and invokes the shared `setup-machine.py` with sudo. Downloads use GitHub HTTPS; checksums detect damaged or mismatched assets, not a compromised release account. The entry scripts themselves must come from the trusted project release. The installation guide uses GitHub's `/releases/latest/download/` asset links and omits version flags. Each script resolves the latest stable release once, then uses that exact tag for the checksum list, shared installer and package. Drafts and prereleases are excluded from automatic selection. A failed `curl` download prevents the chained `bash` command from running; saving the script first also leaves terminal input available for prompts.

To pin a published release, append `--version TAG` to the saved script command, for example `bash install-uno-q.sh --version v0.3.0`. To test a published development prerelease, use `bash install-uno-q.sh --development dev-COMMIT` instead. Replace these example tags with actual published tags, and use the matching option on RetroPie. No GitHub release is created by running an installer.

Option	Behavior
`--version TAG`	Use one exact published release on both machines. Without a tag, select the latest stable GitHub release.
`--development TAG`	Explicitly use a published development prerelease, such as `dev-COMMIT`. This is a release tag, not a branch name.
`--peer HOST`	PowerGlove Vision Controller hostname or IPv4 address for a new RetroPie installation; prompted if omitted in an interactive terminal. Existing destinations are preserved.
`--check`	Use the installed shared checker. No download, package installation, service restart, or helper release. Sudo may be needed to inspect protected settings.
`--help`	Show the entry point options without installing.

Supported baseline: an already provisioned PowerGlove Vision Controller with App Lab CLI **0.13.0**, Arduino sketch platform **1.0.0**, the shipped pinned libraries, and a Debian-based RetroPie installation with Python **3.7 or newer**. The UNO installer rejects other App Lab CLI versions until compatibility has been validated. It does not upgrade the board OS or App Lab. Allow at least 3 GiB of free space in the Arduino home partition (PowerGlove Vision Controller) or `/var/tmp` (RetroPie), and 512 MiB on the system partition for package operations and backups. Larger updates may need more space.

UNO installation stages files in `/home/arduino/ArduinoApps/powerglove-vision`, then uses `arduino-app-cli app start` as the Arduino user to build/upload the sketch and start the app. The root setup phase configures networking, shutdown, default startup, and the user early-start service. It enables Arduino user lingering; it does not issue an early SWD release or reboot during installation. `data/`, generated caches, and the cabinet-only cheat sheet are preserved.

RetroPie installation preserves controller assignments and existing launch hooks. When `~/RetroPie-Setup/retropie_packages.sh` is available for the installing user, setup offers its `install_bin` and `configure` operations for missing RetroArch or FCEUmm. If that setup tool is absent, or its binary installation is unsupported, use RetroPie Setup to prepare the emulator and rerun the installer. No full RetroPie or operating-system installation is attempted.

Registered ROM discovery uses each `/home/*/RetroPie/roms/nes` directory and the configured game registry. Custom ROM locations need manual game checks. For Bad Street Brawler, setup requests consent before changing emulator selection, then invokes the existing game-specific Glove Zap helper. If no core options yet exist, launch and exit FCEUmm once and rerun. Neither installer supplies ROMs, changes cabinet input mergers, nor adds a new RetroPie shutdown mechanism; existing operating-system shutdown controls remain available.

Updates replace managed files and save replaced versions under `/var/backups/powerglove-vision/TIMESTAMP/`, with a `RESTORE.txt`. Backups created by the package installer are root-only. Resolve failures before continuing; the installer stops rather than claiming success. Rerunning is supported. A failed firmware update may require rerunning the previous release through App Lab's normal sketch upload; restoring Linux files alone does not restore firmware. Never restore backup files wholesale over the filesystem root.

Build and publish installation assets

Generate the public PDFs and App Lab ZIP, then build both installer packages:

```
SH
python3 scripts/build-docs-pdf.py
bash scripts/build-app-lab-package.sh
python3 scripts/build-install-packages.py --version dev-COMMIT
```

`output/install/` contains both ZIPs, the two entry scripts, their shared package installer, and `SHA256SUMS`. Package identity and safe paths are validated both at build time and installation time. Private runtime files are excluded.

The **Build installation release** workflow accepts an existing source revision and a new release tag. Its default builds downloadable workflow artifacts only. Select **Publish a GitHub release** explicitly to make the normal download commands available; leave **prerelease** enabled for feature-branch testing. Use a stable release only after physical fresh-device and upgrade validation. Publishing, tagging, or deploying firmware is not implied by building packages.

Installation validation status

Automated tests cover isolated fresh-directory installs and repeat updates, archive rejection, download failure, and settings preservation. These are not fresh-device tests. The new package installer was also run on the existing RetroPie cabinet: receiver, launch hooks, emulator, and Glove Zap checks passed with no technical failures. The PowerGlove Vision Controller full installer still needs an interactive sudo run. A provisioned spare PowerGlove Vision Controller and RetroPie system, real pairing, live gameplay, and a cold boot are required before declaring the installers validated for release. Run read-only checks on both devices after installation; no automated check proves that a hand gesture controls a game correctly.

Shared installer and application sources

Both installer entry points are generated from `scripts/templates/install.sh.in`. Run `python3 scripts/build-installer-scripts.py` after editing that template; use `--check` to detect drift without changing files.

`scripts/application-payload.py DESTINATION` stages the public application into an empty directory. Both App Lab packaging and Wi-Fi deployment use this selection, excluding private settings, artwork masters, and generated installer archives. `config/profiles.json` is release-owned and is backed up and replaced during updates, so tested shared defaults and new fields take effect consistently. Runtime calibration, pairing, tuning, and cabinet settings remain in place.

Installation ownership manifest

`scripts/installation-manifest.py` manages the application payload in `/home/arduino/ArduinoApps/powerglove-vision` on PowerGlove Vision Controller and `/opt/powerglove-src` on RetroPie. Package installation and Wi-Fi deployment use the same implementation. Host service units, launch hooks, controller assignments, and system configuration remain under their existing installers; the payload manifest does not prune them.

Each root contains `.powerglove-install.json` with format version 1, the absolute installation root, release identity, and relative paths with SHA-256 hashes and permission modes. Private data, caches, and the local `docs/cheatsheet.md` are excluded from managed ownership. `config/profiles.json` is deliberately managed and replaced from the release; its previous copy is retained in the update backup.

On the first manifest-enabled update, incoming package paths are installed using the existing backup-and-replace behavior. Unknown files absent from the package are left alone: the installer does not infer an old inventory from the directory. Subsequent updates back up and remove obsolete files only when their bytes and permissions still match the previous manifest. Local changes, including permission changes, are retained and reported, whether or not the new package contains that path. Their old baseline remains recorded so later updates cannot silently adopt or delete them. Restore the previous installed version of a locally modified file before retrying if you want the installer to replace it with the new release.

Updates take an exclusive lock, validate all paths before writing, back up changed and removed files, and publish the new manifest last. A failed write rolls back the payload. A process interruption leaves `.powerglove-install-pending.json`; another update refuses to proceed until recovery. Backups include the old manifest, changed files, `transaction.json`, and `RESTORE.txt`. An installation failure after payload staging (for example, host setup or App Lab startup) does not undo a successfully committed payload; use the reported backup or previous release.

The normal installer `--check` reports missing or modified managed files and an unfinished transaction without changing anything. For payload-only checks:

```
SH
python3 scripts/installation-manifest.py /home/arduino/ArduinoApps/powerglove-vision --check
```

For an interrupted update, stop the application, then use the script from a trusted release staging directory (the installed copy may have been interrupted):

```
SH
sudo python3 scripts/installation-manifest.py /home/arduino/ArduinoApps/powerglove-vision --recover
```

On RetroPie, use `/opt/powerglove-src` and run recovery with `sudo`. Restart and run installer checks afterward. Do not remove the pending journal or manifest to bypass recovery. The manifest never scans or deletes unknown user files or directories. `--source STAGING --backup BACKUP` applies a staged payload; BACKUP must be a new location outside both source and installation trees. This is used by the deployment script; normal users should use the two standard installers.

Optional native latency diagnostics

These developer tools do not change recognition settings or production packet and native-state formats. See the complete session procedure for camera placement, interpretation, process activation, and rollback.

Interface	Default	Meaning
`POWERGLOVE_DIAGNOSTIC_TRACE`	Unset/off	Private output prefix inherited by the vision worker or receiver at process startup. Writes `PREFIX.ROLE.PID.json`, never overwriting a file.
`POWERGLOVE_DIAGNOSTIC_SECONDS`	`180`	Finite trace duration, 1-600 seconds from initialization. Python and diagnostic core buffers hold at most 20,000 events each.

Interface	Default	Meaning
`POWERGLOVE_BUILD_DIAGNOSTICS=1`	Unset/off	Build a separately named diagnostic core in a fresh directory; no installation or launch-selection change.
`POWERGLOVE_CORE_DIAGNOSTIC_TRACE`	Unset/off	New private CSV path used only by the diagnostic core. Export occurs at normal game unload.

`prepare-end-to-end-session.py` performs a read-only Controller/RetroPie/source preflight. Its `prepare` phase allows an intentionally stopped camera/game; its `record` phase requires live gameplay readiness. It writes only an allowlisted, privacy-safe manifest to a new private directory.

`run-native-latency-session.py` takes `--status-url`, a new `--output-dir`, and `--protocol smoke|full`; it reads status and provides operator cues. An optional `--preflight` records only the preflight file's SHA-256 in the session manifest. Its default `--poll-interval 0.25` avoids treating cached duplicates as additional inferences or needlessly loading the Controller web service. `manage-latency-traces.py start` applies bounded Controller and receiver tracing, and `stop` restores their production environments before collecting finalized files. Its private state file is required for restoration. It does not select a diagnostic core. `analyze-latency-trace.py` takes `--controller`, `--receiver`, optional `--core`, and a new `--output`; core joins require `--same-cabinet-boot` after verifying the same boot/native-state path. `benchmark-diagnostic-overhead.py --output PATH` measures local synthetic trace overhead without controlling hardware.

`analyze-latency-video.py` takes `--video`, a new `--output-dir`, and optional `--frames` (up to 500 zero-based indexes), `--around`/`--radius`, or `--annotations`. Optional `--capture-fps` records a confirmed camera mode and suggests the playback-to-capture scale without approving it. An initial run creates an overview contact sheet and an unapproved `smoke` or `full` annotation template; null event frames and `timing_verified: false` prevent accidental acceptance. It needs PyAV and Pillow only in a temporary environment on the analysis computer. Stored display rotation is applied to review images while presentation timestamps remain unchanged. It requires an exact video hash and reviewed timing before measuring annotated events. All raw traces, recordings, and position annotations remain temporary and local. Public status still samples inference; trace joins never estimate cross-host network delay by subtracting independent monotonic clocks.

Historical bounded native X/Y speed curve

The former Super Glove Ball experiment used consecutive selected coordinates and their capture timestamps to measure hand speed. The selected point comes directly from validated MediaPipe geometry and is clamped to calibrated reach before filtering. Speed is normalized independently by directional X/Y reach, then combined into one vector magnitude and one follow weight. Calibrated X/Y jitter forms an elliptical resting region with entry/exit hysteresis and a small fixed fallback for older calibrations. Above that region, `motion_slow_follow` rises smoothly to one-to-one response at `motion_full_speed`. A meaningful reversal adopts the newest real coordinate; a stop settles inside measured noise immediately and exactly by the next fresh result. The weight is always capped at `1.00`, so the filter cannot predict or overshoot. This curve is no longer selectable; the live path uses Latest coordinate with the same validation, edge clamp, calibration, reach mapping, gestures, and safety.

Older configurations containing `motion_coordinate_boost` or `motion_coordinate_max` remain loadable. They are retained for historical comparison, but a maximum above `1.00` no longer extrapolates native output. Use `benchmark-vision-replay.py` followed by `benchmark-native-motion-curve.py` to compare the old deployed experiment, its safely capped form, the bounded speed curve, and unsmoothed latest coordinates. See offline analysis.